

Lecture Note 5: ESP32 – Web Servers and Clients

CSS452 Internet of Things

Dr. Seksan Laitrakun

School of ICT, SIIT, Thammasat University

To learn how to use the ESP32's Wi-Fi capabilities to **create interactive, web-based IoT applications**. We will build the following projects:

- **ESP32 as a Web Server**: We will create a local web page, hosted on the ESP32, to remotely control outputs and display live sensor data from your browser.
- **ESP32 as a Web Client**: We will send data from the ESP32 to a cloud IoT platform, allowing you to monitor your sensors from anywhere in the world.

Lecture Outline::

1. Introduction to a Web Server
2. Basic HTML and CSS
3. ESP32 as a Local Web Server to Control Actuators
4. ESP32 as a Local Web Server to Monitor Sensor Data
5. ESP32 as a Client of Web Services
6. Advanced Topics for ESP32 Web Servers and Clients

- Random Nerd Tutorials: <https://randomnerdtutorials.com/projects-esp32/>
- Last Minute Engineers: <https://lastminuteengineers.com/esp32-arduino-ide-tutorial/>
- Thai Easy Elect (Thai Language): <https://blog.thaieasyelec.com/espino32-article-table-of-contents/>

1. Introduction to a Web Server

This section consists of

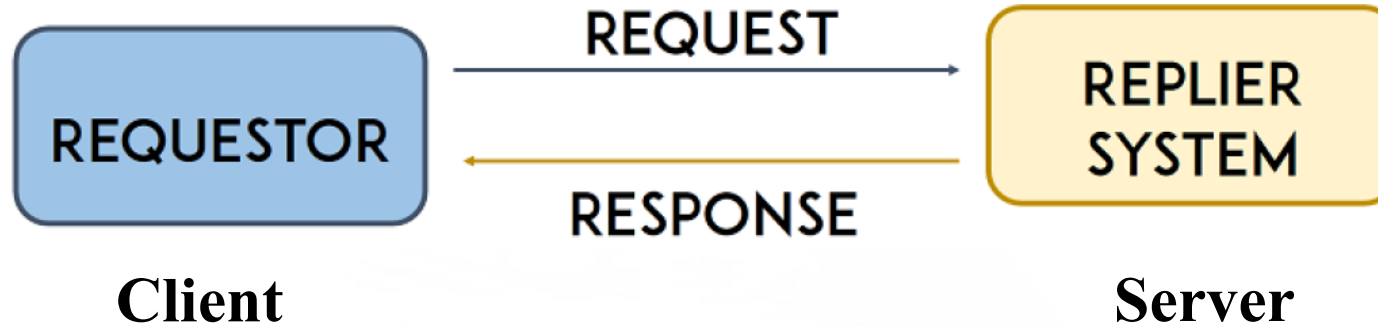
1.1 Client-Server Architectures

1.2 HTTP Transmission Protocol

1.1 Client-Server Architectures

1. Request-response

This is the fundamental communication pattern for the web.



It's a simple, three-step process:

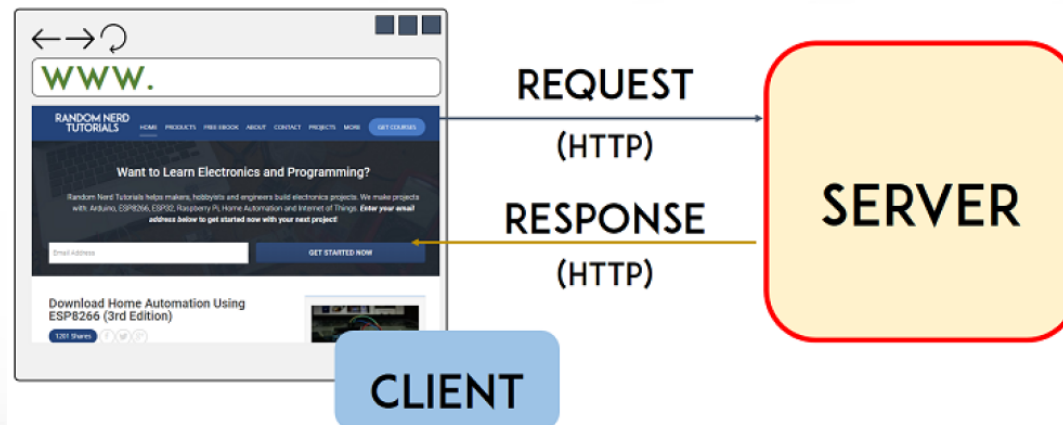
- 1) **Request**: A client (like your web browser) sends a **request message** to a server.
 - "GET /homepage.html"
- 2) **Process**: The server receives the request and processes it.
 - Finds the requested file.
- 3) **Response**: The server sends a **response message** back to the client.
 - Sends the HTML code for the homepage.

1.1 Client-Server Architectures

Ex: How Your Browser Gets a Web Page

When you type a URL and press the Enter, a simple four-step process happens in the background:

- 1) **You Start:** You type a URL into your web browser.
- 2) **The Request:** Your browser (the Client) sends an **HTTP Request** to the web server.
 - “Please give me the file for this web page.”
- 3) **The Response:** The Server finds the requested files (HTML, CSS, etc.) and sends an **HTTP Response** back to your browser.
 - “Here are the files you asked for.”
- 4) **The Result:** Your browser receives the files and renders the web page on your screen.

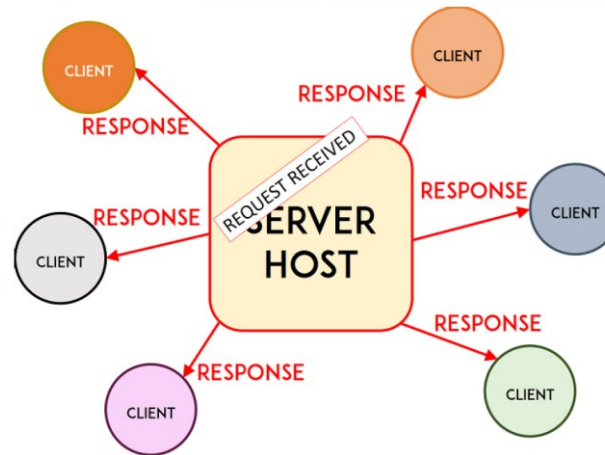


1.1 Client-Server Architectures

2. Web Server

What is a **Web Server**?

- a) A program that continuously listens for incoming **HTTP requests** from clients (like a web browser).
- b) When a request is received, it processes it and sends back an **HTTP response** (like an HTML page).

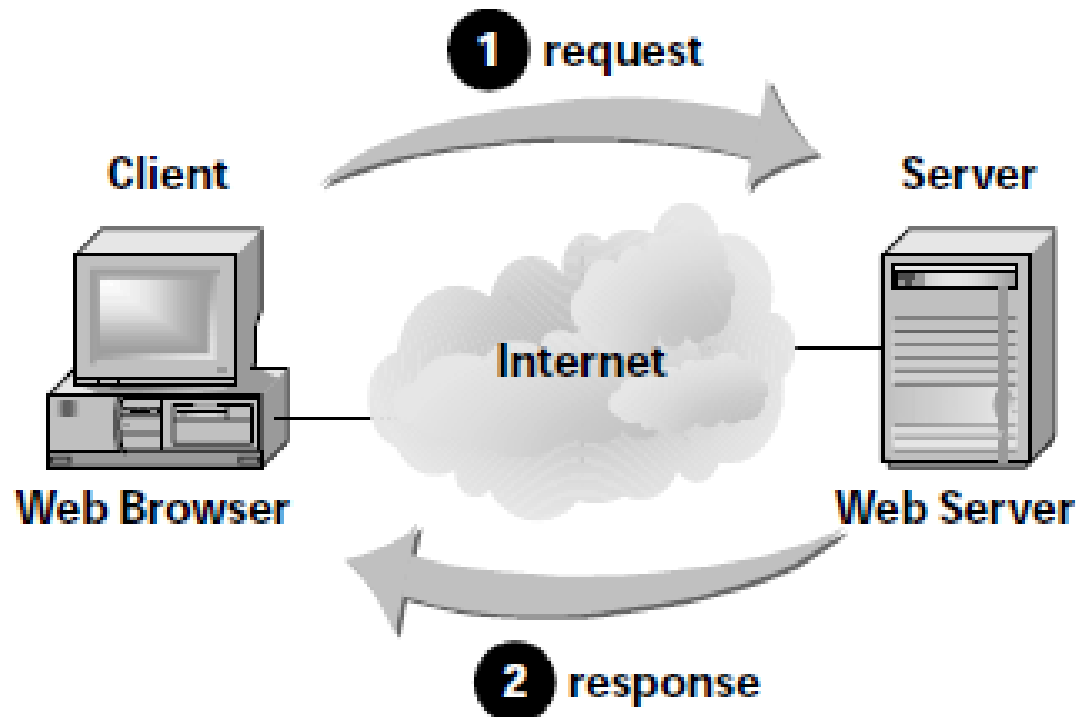


1.2 HTTP Transmission Protocol

1. Process

Standard procedures when we open a website (the computer we use to open a website is called a client; the computer that stores the html source code is called a server):

- 1) The client sends a **HTTP request message** to the server.
- 2) The server sends a **HTTP response message** (including a HTML page) to the client.



1.2 HTTP Transmission Protocol

2. HTTP Request Message Format

The web server will receive the **HTTP request message** from the client. Its format is shown below

```
METHOD /path-to-resource HTTP/version-number  
Header-Name-1: value  
Header-Name-2: value  
a blank line  
[optional request body]
```

where

- METHOD = a request method, e.g., get or post.
- The method “**get**” will be the method we use (similar to type a URL).
- /path-to-resource = URL path after the host website name.
- Header-Name-1 = Host; value = the host website name
- Read more from:

http://www.w3schools.com/Tags/ref_httpmethods.asp

1.2 HTTP Transmission Protocol

2. HTTP Request Message Format

Example: If you type www.example.com/index.html on your web browser and, then, press enter. Your computer will work as a client and sends the following HTTP request message to the server:

```
GET /index.html HTTP/1.1  
Host: www.example.com
```

The HTTP request message itself that is sent out will be one line of texts (to represent the above HTTP request message):

```
GET /index.html HTTP/1.1\r\nHost: www.example.com\r\n\r\n
```

Note: \r = carriage return; \n = go to the new line.

1.2 HTTP Transmission Protocol

3. HTTP Response Message

The server will send the **HTTP response message** to the client. The client's web browser will read this message and display the corresponding web page. The format of the HTTP response message is shown below.

```
HTTP/version-number  status-code  message
Header-Name-1:  value
Header-Name-2:  value
A blank line
[HTML Body]
```

Examples of status-code and message are:

- 200 OK
- 400 Bad Request
- 404 Not Found

Read more from:

https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

1.2 HTTP Transmission Protocol

3. HTTP Response Message

Example: If you type

www.example.com/index.html

on your web browser and, then, press enter. An example of the HTTP response message (sent back from the web server) is shown here. The web browser will display the web page corresponding to the HTML body.

HTTP/1.1 200 OK

Date: Mon, 23 May 2005 22:38:34 GMT

Content-Type: text/html; charset=UTF-8

Content-Encoding: UTF-8

Content-Length: 138

Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT

Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)

ETag: "3f80f-1b6-3e1cb03b"

Accept-Ranges: bytes

Connection: close

<html>

<head>

<title>An Example Page</title>

</head>

<body>

Hello World, this is a very simple HTML document.

</body>

</html>

2. Basic HTML and CSS

In this section, we will build a basic web page with step-by-step instructions, so that we can easily change how it looks. This section consists of

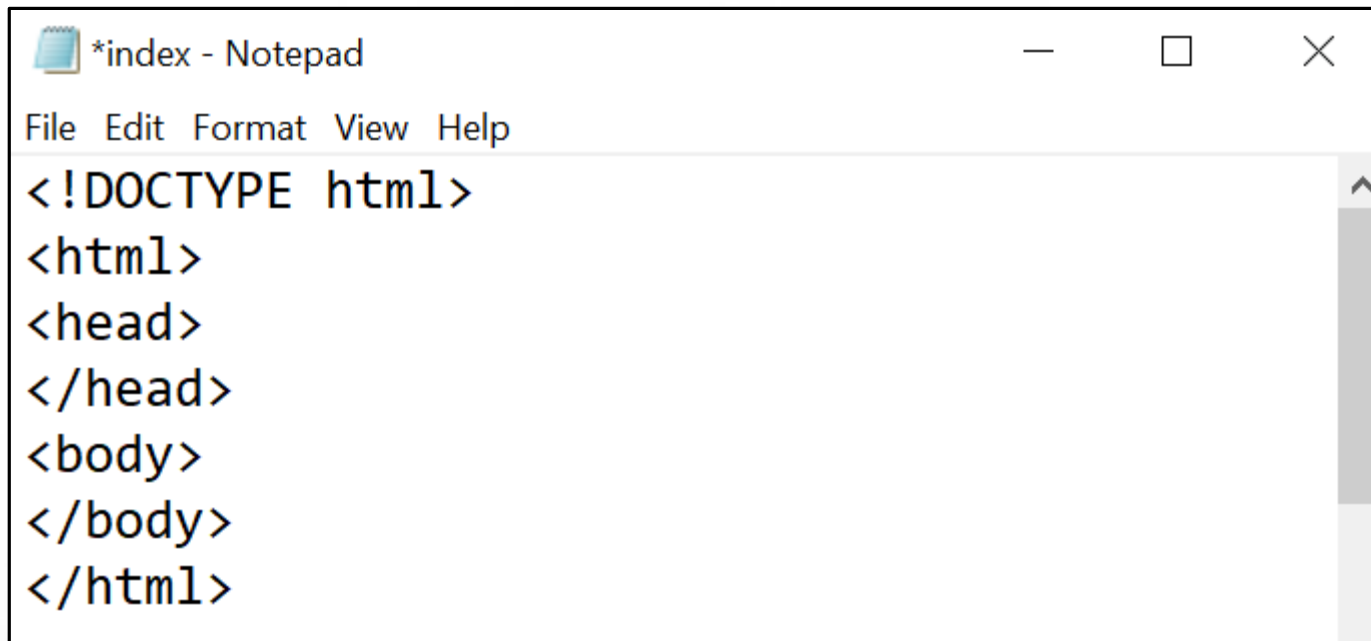
2.1 Basic HTML

2.2 Basic CSS

2.1 Basic HTML

1. Introducing HTML

- HTML stands for **Hyper Text Markup Language**.
- It is the standard language used to structure the content of a web page.
- HTML uses tags (e.g., <p>, <h1>) to define different elements like paragraphs and headings.
- A web browser's job is to read the HTML file and render the content visually based on these tags.



A screenshot of a Notepad window titled '*index - Notepad'. The window contains the following HTML code:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
</body>
</html>
```

2.1 Basic HTML

1. Introducing HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My Web Page</title>
```

```
</head>
```

```
<body>
```

```
<h1>Hello, World! </h1>
```

```
<p>This is the content you can see. </p>
```

```
</body>
```

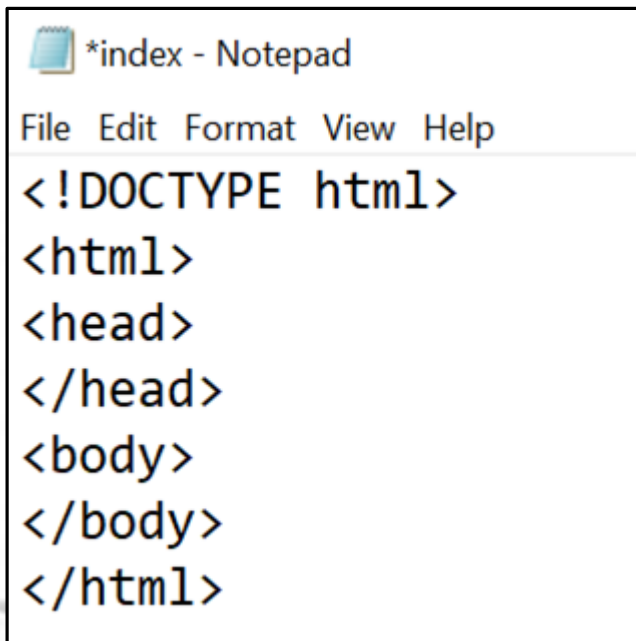
```
</html>
```

- The first line of any HTML document is always `<!DOCTYPE html>`. This tells the web browser this document is an HTML file.
- The structure of the web page should go between the `<html>` and `</html>` tags. The `<html>` tag indicates the beginning of a web page and the `</html>` tag indicates the end of the page.
- The HTML document is divided into two main parts: the head and the body. The head goes within the `<head>` and `</head>` tags and the body within the `<body>` and `</body>` tags.
- The head is where you insert data about the HTML document that is not directly visible to the end user, but adds functionalities to the web page like the title, scripts, styles and more – this is called metadata.
- The body includes the content of the page like headings, text, buttons, tables, etc.

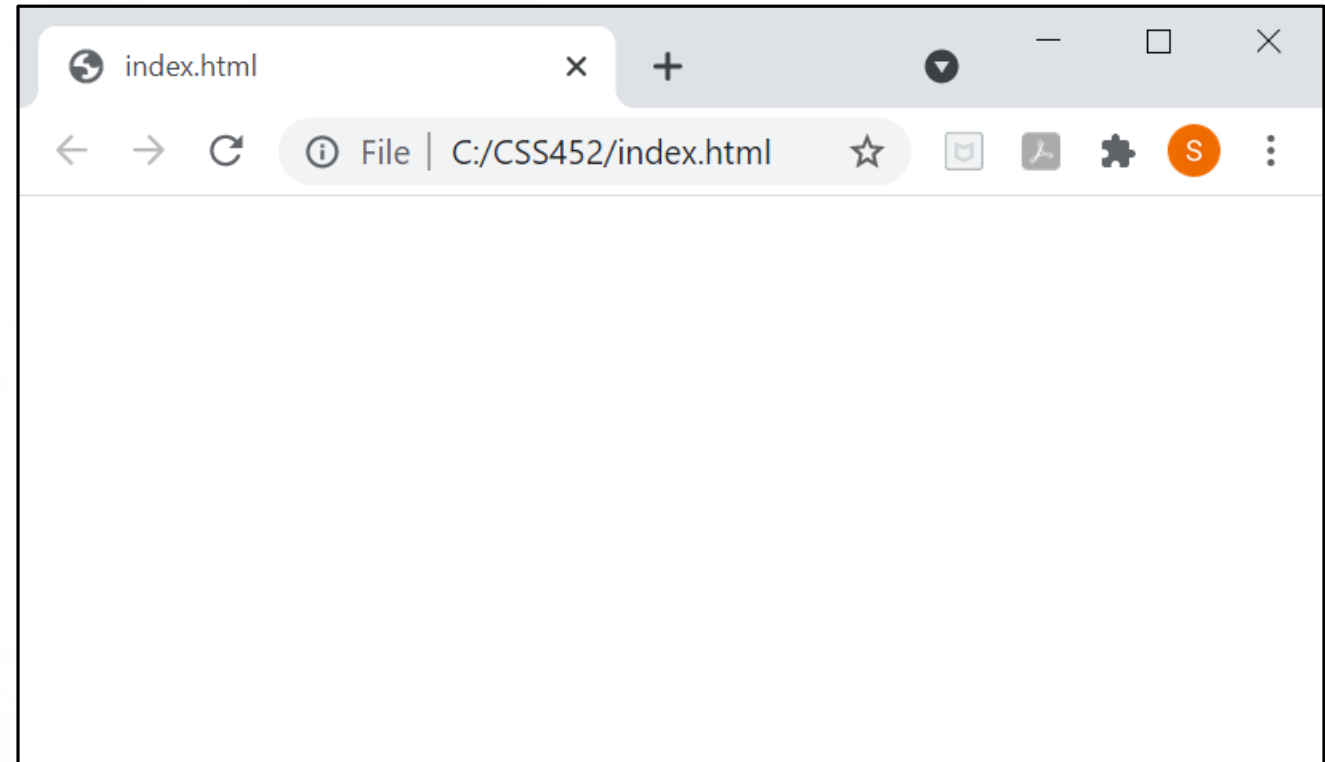
2.1 Basic HTML

Ex: Create a HTML File

- 1) Open a Text Editor program (you can use any text editor you like, we use Notepad) and write the HTML texts below.
- 2) Save the file as index.html.
- 3) Open your browser and drag the HTML file to a browser tab.
- 4) You will just see a blank page because you haven't added anything to the HTML file yet.



```
*index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
</head>
<body>
</body>
</html>
```



2.1 Basic HTML

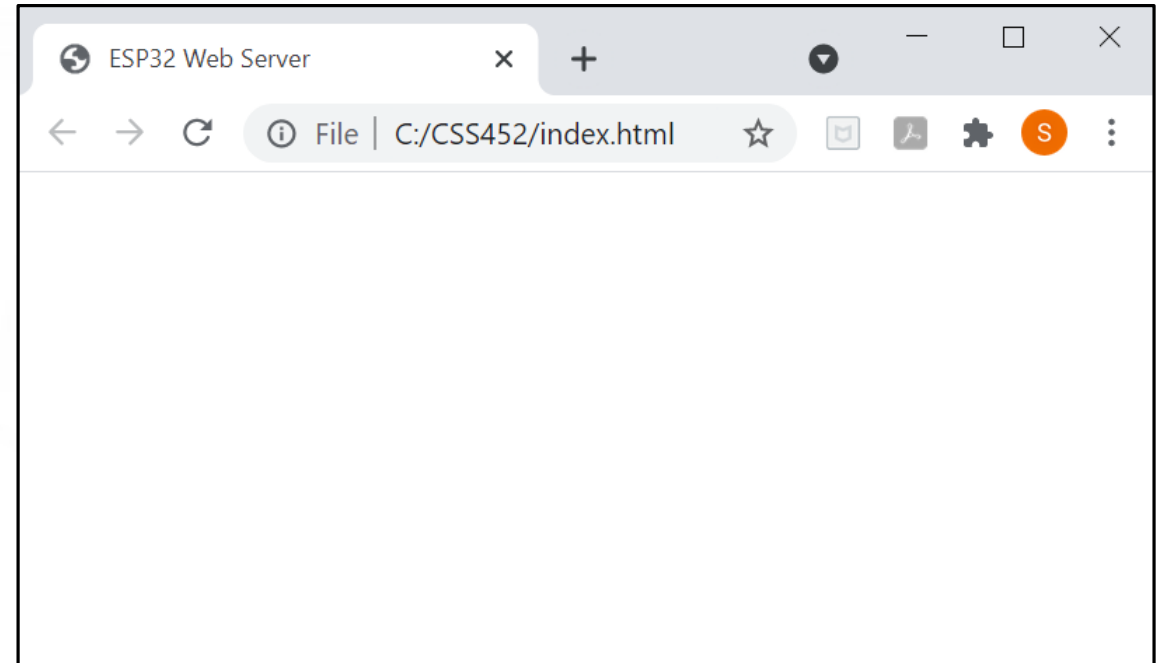
Ex: Create a Title

The title of your web page is the text that shows in the web browser tab.

- The web page title should go between the `<title>` and `</title>` tags, that should go between the `<head>` and `</head>` tags.
- Add a **title to your web page** by typing the title between `<title>` and `</title>` tags, as shown in the example below.



```
index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
</head>
<body>
</body>
</html>
```



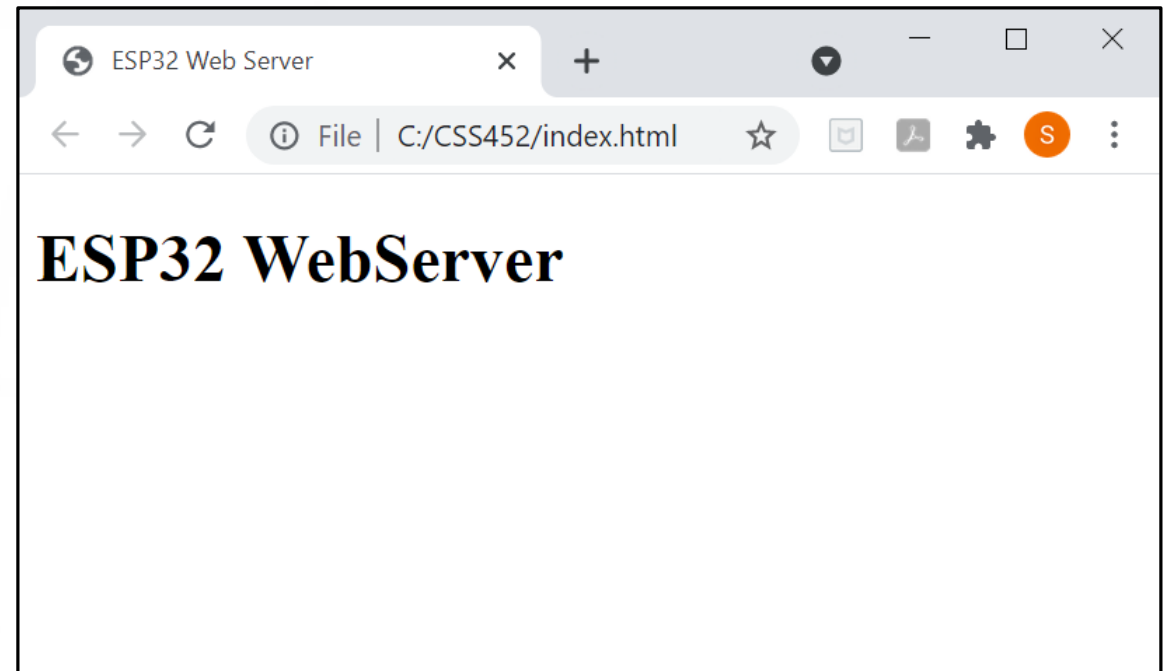
2.1 Basic HTML

Ex: Create a Heading

Headings are used to structure the text on the web page.

- Headings begin with an h followed by a number that indicates the heading strength.
- For example `<h1>` and `</h1>` are the tags for heading 1, `<h2>` and `</h2>` for heading 2, until heading 6.
- The heading tags should be between the `<body>` and `</body>` tags. Add some headings to your document. You can use the following text as a reference.

```
index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
</head>
<body>
  <h1>ESP32 WebServer</h1>
</body>
</html>
```

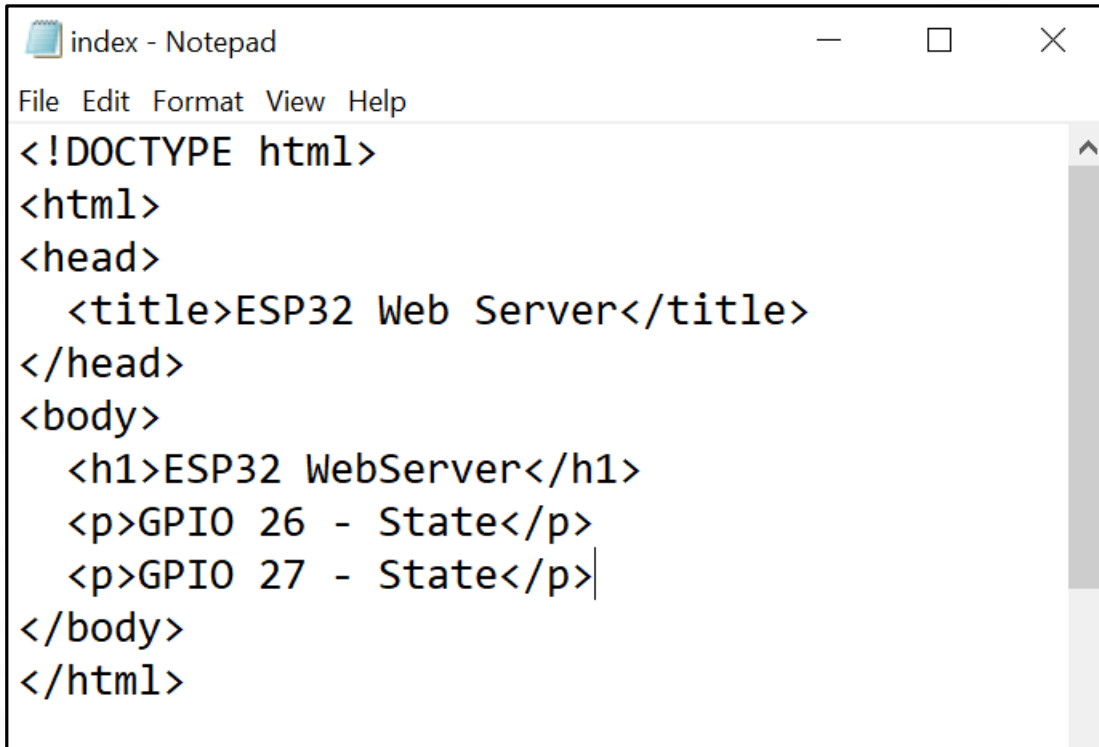


2.1 Basic HTML

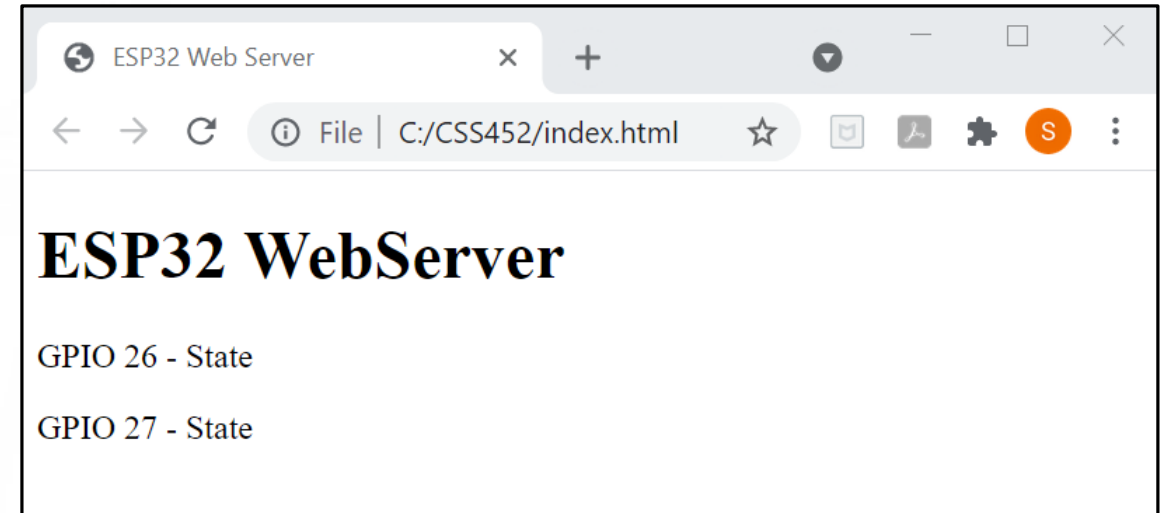
Ex: Create a Paragraph

The paragraphs are used to place text.

- Every paragraph should go between the `<p>` and `</p>` tags. Add some paragraphs to show the state of GPIO 26 and GPIO 27.



```
index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
</head>
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p>GPIO 27 - State</p>
</body>
</html>
```



2.1 Basic HTML

Ex: Create a Button

How to create a button: Use the `<button>` tag.

- The text between `<button>` and `</button>` becomes the label on the button's face.

Example Code:

```
<p>GPIO 26: <button>ON</button> <button>OFF</button></p>  
<p>GPIO 27: <button>ON</button> <button>OFF</button></p>
```

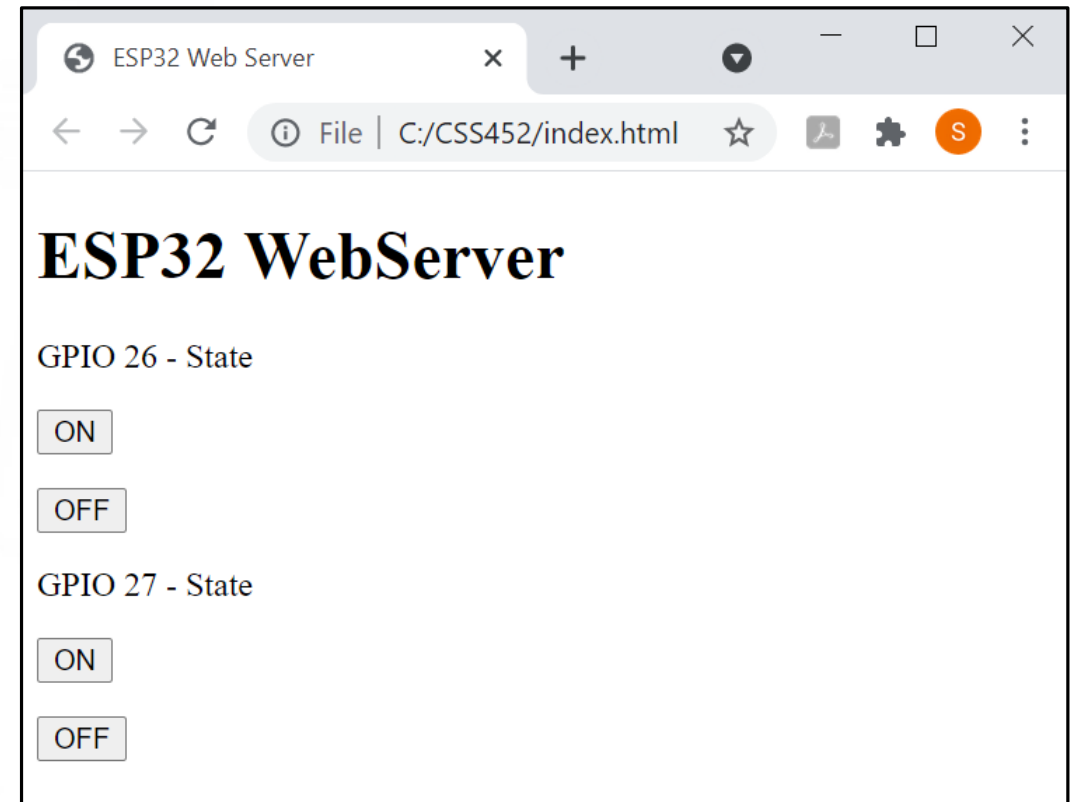
2.1 Basic HTML

Ex: Create a Button

Here is an example.

- Note that when we click on the buttons, nothing happens because those buttons do not have any hyperlink associated with them.
- We need to add hyperlinks to the buttons.

```
index - Notepad
File Edit Format View Help
<html>
<head>
  <title>ESP32 Web Server</title>
</head>
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><button>ON</button></p>
  <p><button>OFF</button></p>
  <p>GPIO 27 - State</p>
  <p><button>ON</button></p>
  <p><button>OFF</button></p>
</body>
</html>
```



2.1 Basic HTML

Ex: Create a Hyperlink

HTML links are called hyperlinks. You can add hyperlinks to text, images, buttons, or any other HTML element.

- 1) To add a hyperlink you use the `<a>` and `</a>` tags, in the following format:

```
<a href="url">element</a>
```

- 2) Between the `<a>` and `</a>` tags, you should place the **HTML element** you want to apply the link to. For example, to apply the link to one of the “OFF” buttons:

```
<a href="url"><button>OFF</button></a>
```

- 3) The href attribute specifies where the link should go. When you click the GPIO 26 ON button, you want to be redirected to the root page followed by /26/on. To do that, you should add that URL to the href attribute, as follows:

```
<a href="/26/on"><button>ON</button></a>
```

- 4) When you click the GPIO 26 OFF button, you want to redirect to /26/off:

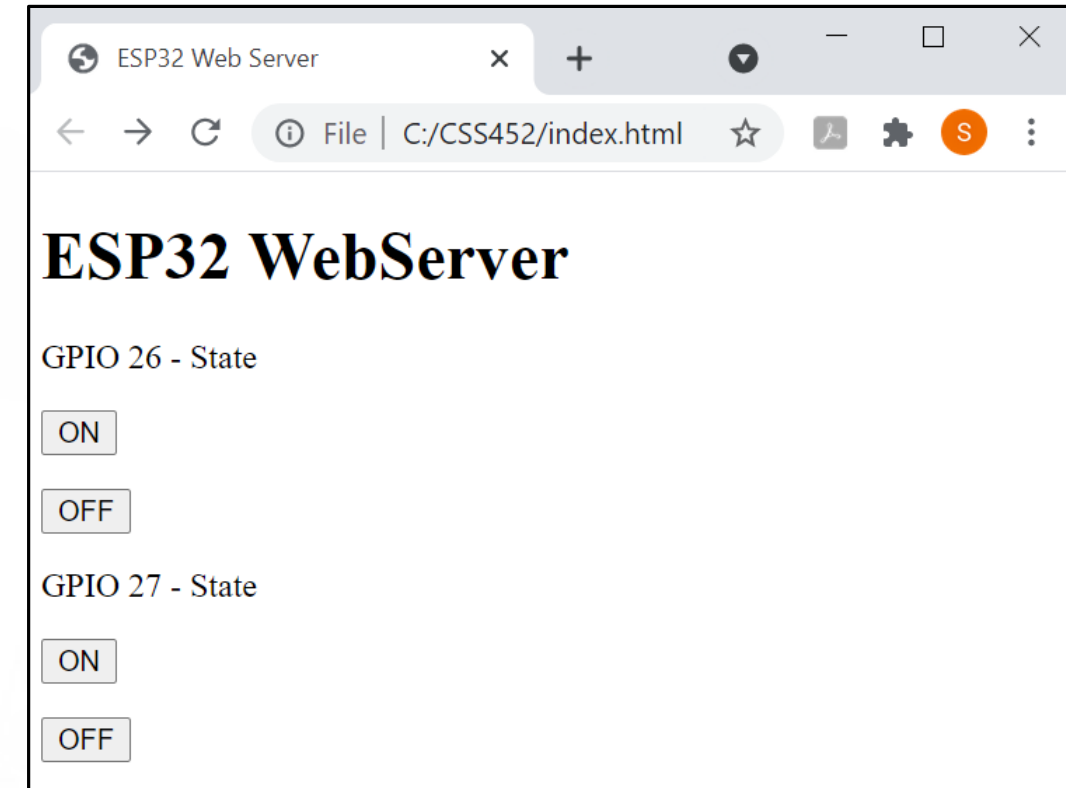
```
<a href="/26/off"><button>OFF</button></a>
```

2.1 Basic HTML

Ex: Create a Hyperlink

You should add the appropriate hyperlink to each of your buttons as follows:

```
index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
</head>
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><a href="/26/on"><button>ON</button></a></p>
  <p><a href="/26/off"><button>OFF</button></a></p>
  <p>GPIO 27 - State</p>
  <p><a href="/27/on"><button>ON</button></a></p>
  <p><a href="/27/off"><button>OFF</button></a></p>
</body>
</html>
```

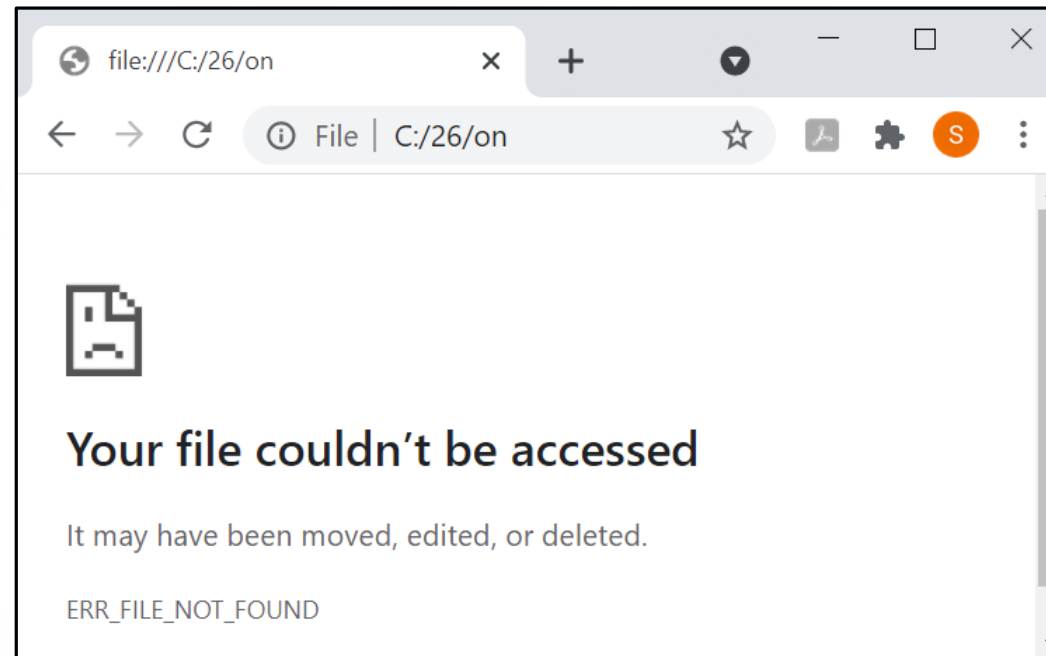


2.1 Basic HTML

Ex: Create a Hyperlink

At the moment, you get the error “file was not found” because you do not have any file to that URL.

- Later, this will be solved on the Arduino IDE, because your ESP32 will send different HTML text when you click on the buttons.



2.2 Basic CSS

1. Syntax

CSS stands for **Cascading Style Sheets** and it is used to describe how the elements in a web page look. It describes a certain part of the page like a particular tag or a particular set of tags. The CSS can be added to the HTML file or in a separate file that is referenced by the HTML file. We are going to add the CSS to the HTML file because it is easier to add to the Arduino IDE.

- 1) When added to the HTML file, the CSS should go between the **<style>** and **</style>** tags, that should go in the head of the HTML file.

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <style>YOUR CSS GOES HERE</style>
</head>
```

2.2 Basic CSS

1. Syntax

- 2) CSS uses selectors to style your HTML content. The selector points to the HTML element you want to style. Selectors have properties, which in turn have values. The style for a certain selector should go between curly brackets {}. The value is attributed to a property using a colon (:). Every value should end with a semicolon (;). Each selector can have, and normally does have, more than one property.

```
selector {  
    property: value;  
}
```

2.2 Basic CSS

2. Styling the Page

If we want to set to the html style which will be applied to the whole web page i.e., all web page content goes between the <html> and </html> tags, we can design it by using the html selector. For example, we define the font-family to Helvetica, the content is displayed as a block, you set 0px for the margins and align all the page at the center using “auto”. The html selector will be defined as:

```
html {  
    font-family: Helvetica;  
    display: inline-block;  
    margin: 0px auto;  
    text-align: center;  
}
```

2.2 Basic CSS

```
index - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <style>
    html { font-family: Helvetica;
           display: inline-block;
           margin:0px auto;
           text-align: center;
         }
  </style>
</head>
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><a href="/26/on"><button>ON</button></a></p>
  <p><a href="/26/off"><button>OFF</button></a></p>
  <p>GPIO 27 - State</p>
  <p><a href="/27/on"><button>ON</button></a></p>
  <p><a href="/27/off"><button>OFF</button></a></p>
</body>
</html>
```

We add the html selector within the `<style>` tags.



Next, we style the buttons that we use.

- 1) We create a class attribute for a button. The syntax of a class attribute is

`<element class="classname">`

Here, the element is button. The classname for ON will be named button while the classname for OFF will be button and button2 as shown below.

`<p><a href="/26/on"><button class="button">ON</button></a></p>`

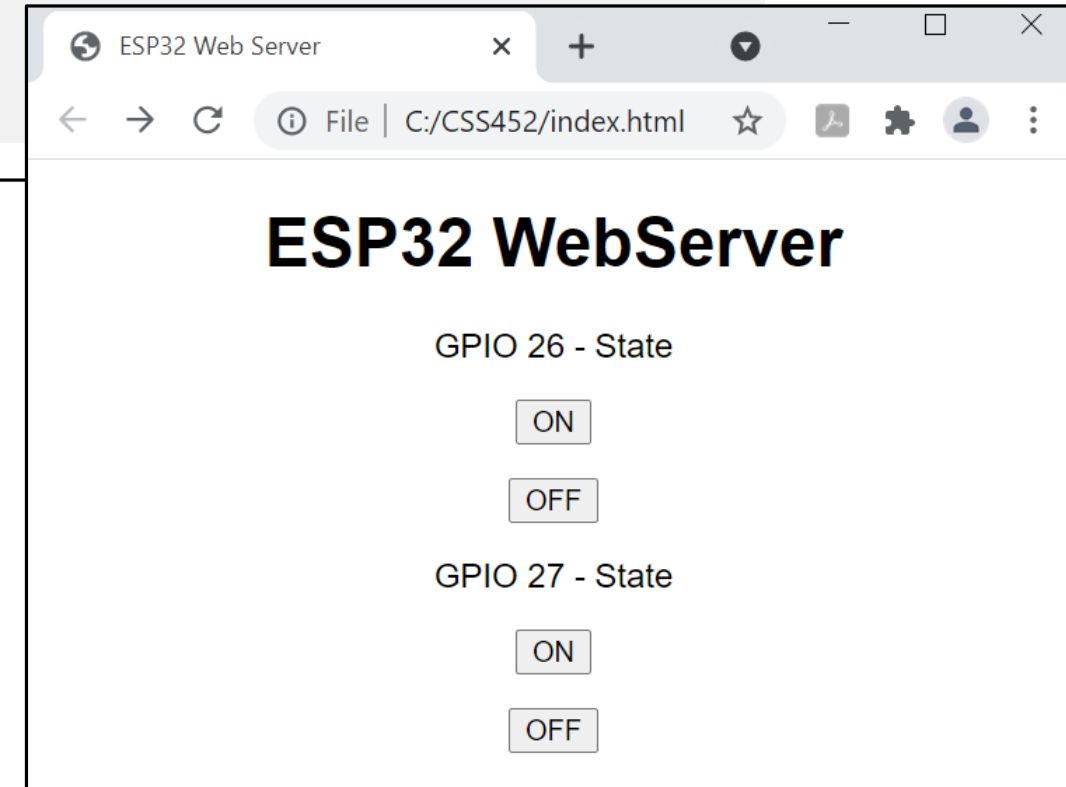
`<p><a href="/26/off"><button class="button button2">OFF</button></a></p>`

The buttons for GPIO26 will be designed the same way.

2.2 Basic CSS

3. Styling the Button

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <style>
    html { font-family: Helvetica;
           display: inline-block;
           margin:0px auto;
           text-align: center;
         }
  </style>
</head>
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><a href="/26/on"><button class="button">ON</button></a></p>
  <p><a href="/26/off"><button class="button button2">OFF</button></a></p>
  <p>GPIO 27 - State</p>
  <p><a href="/27/on"><button class="button">ON</button></a></p>
  <p><a href="/27/off"><button class="button button2">OFF</button></a></p>
</body>
</html>
```



There is nothing changed since we did not define the style of the class name button and button2.

2.2 Basic CSS

3. Styling the Button

2) To specify the style of the classname button, we add the following text into the index.html.

```
.button {  
    background-color: #4CAF50;  
    border: none;  
    color: white;  
    padding: 16px 40px;  
    text-decoration: none;  
    font-size: 30px;  
    margin: 2px;  
    cursor: pointer;  
}
```

- The background-color property, as the name suggests, defines the button's background color. The colors can be set by using their name – HTML recognizes basic color names – or by using the hexadecimal or RGB color code – search on the web for “hexadecimal color picker” to search for a hexadecimal reference for a specific color. Here we're using hexadecimal color code.
- We set the border property to none, and the button text to white.
- The padding defines a space around the button – in this case we set 16px by 40px.
- We set the text decoration to none, font size of 30px, margin of 2px, and the cursor to a pointer – this will change the cursor to a pointer when you drag the mouse over the button.

- 3) To specify the style of the classname button2, we add the following text into the index.html.

```
.button2 {  
    background-color: #555555  
}
```

Here, we only define the color (grey) to the classname button2.

The off button belongs to classname “button” and classname “button2”, so it will have properties from both.

We add these two CSS styles for the button elements within the <style> tags in the index.html and refresh the webpage.

2.2 Basic CSS

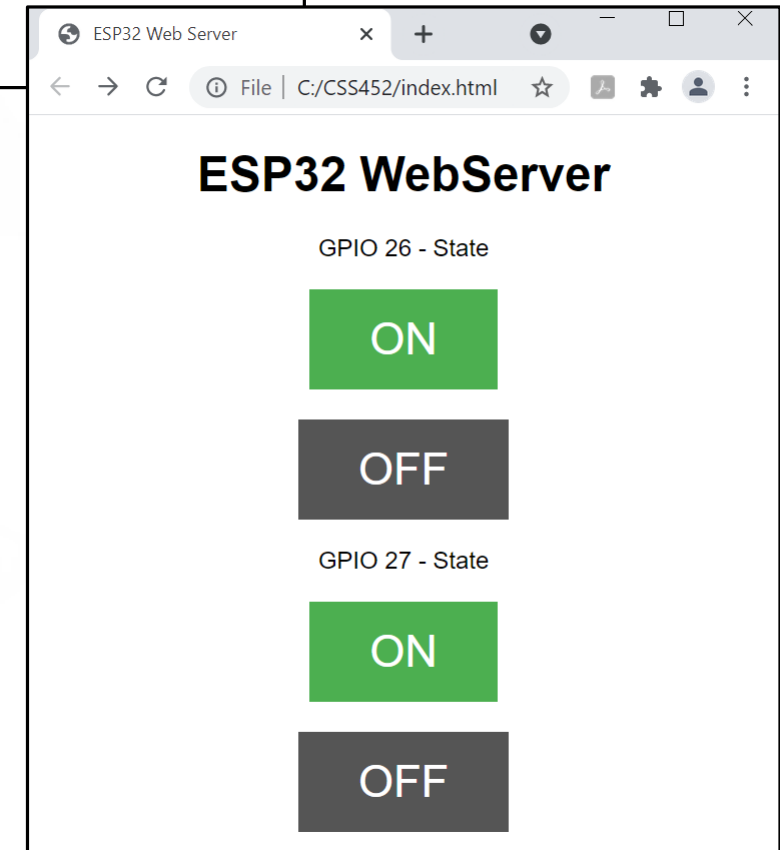
3. Styling the Button

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <style>
    html { font-family: Helvetica;
           display: inline-block;
           margin: 0px auto;
           text-align: center;
         }
    .button { background-color: #4CAF50;
              border: none;
              color: white;
              padding: 16px 40px;
              text-decoration: none;
              font-size: 30px;
              margin: 2px;
              cursor: pointer;
            }
    .button2 { background-color: #555555;
              }
  </style>
</head>
```

2.2 Basic CSS

```
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><a href="/26/on"><button class="button">ON</button></a></p>
  <p><a href="/26/off"><button class="button button2">OFF</button></a></p>
  <p>GPIO 27 - State</p>
  <p><a href="/27/on"><button class="button">ON</button></a></p>
  <p><a href="/27/off"><button class="button button2">OFF</button></a></p>
</body>
</html>
```

After updating the code and saving it, we refresh the webpage and get a new result as shown here.



2.2 Basic CSS

4. Metadata

In the head of the index.html file, under the <title> tag, we add the following two lines.

- 1) we also add the <meta> tag that makes your web page responsive in any web browser (including mobile web browser).

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

The <meta> tag provides metadata about the HTML document. Metadata will not be displayed on the page, but provides useful information to the browser such as how to Display the content.

- 2) We also add the following line:

```
<link rel="icon" href="data:,">
```

To prevent requests from the browser to the ESP32 about the favicon. The favicon is the shortcut icon that appears on the web browser tab.

Then, our complete index.html file is shown in the next page.

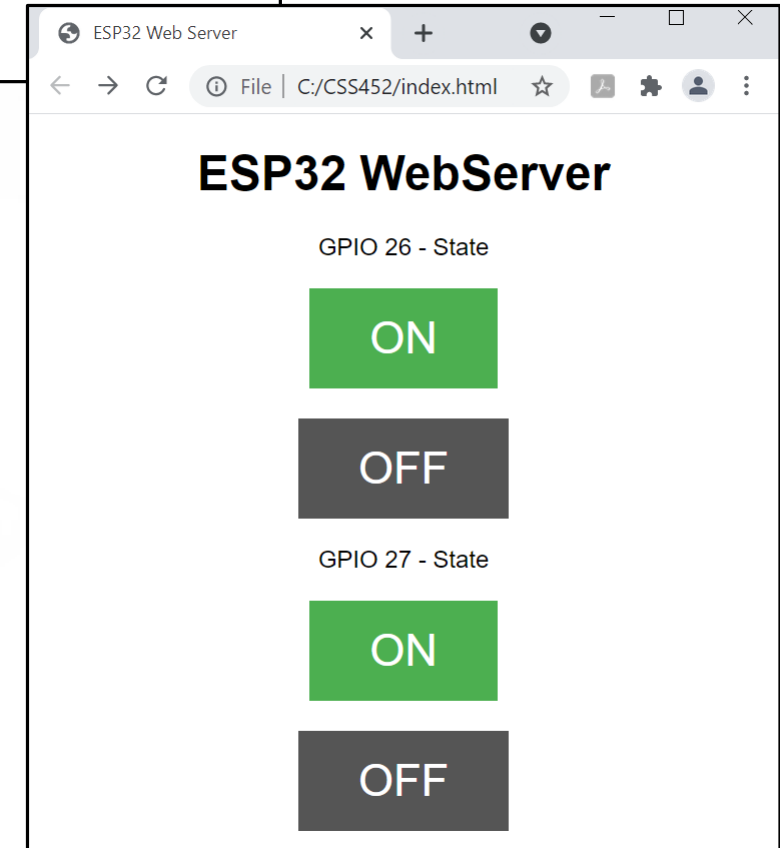
2.2 Basic CSS

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <meta name="viewport" content="width=device-width, initialscale=1">
  <link rel="icon" href="data:,">
  <style>
    html { font-family: Helvetica;
           display: inline-block;
           margin: 0px auto;
           text-align: center;
         }
    .button { background-color: #4CAF50;
              border: none;
              color: white;
              padding: 16px 40px;
              text-decoration: none;
              font-size: 30px;
              margin: 2px;
              cursor: pointer;
            }
    .button2 { background-color: #555555;
              }
  </style>
</head>
```

2.2 Basic CSS

```
<body>
  <h1>ESP32 WebServer</h1>
  <p>GPIO 26 - State</p>
  <p><a href="/26/on"><button class="button">ON</button></a></p>
  <p><a href="/26/off"><button class="button button2">OFF</button></a></p>
  <p>GPIO 27 - State</p>
  <p><a href="/27/on"><button class="button">ON</button></a></p>
  <p><a href="/27/off"><button class="button button2">OFF</button></a></p>
</body>
</html>
```

After updating the code and saving it, we refresh the webpage and get a new result as shown here.



3. ESP32 as a Local Web Server to Control Actuators

In this section, we will learn how to create a simple web server with the ESP32 to control outputs. The web server we will build can be accessed with any device that has a browser: smartphone, tablet, laptop, on the local network. This section consists of

3.1 Building ESP32 as a Local Web Server

3.2 ESP32 Web Server to Control Outputs

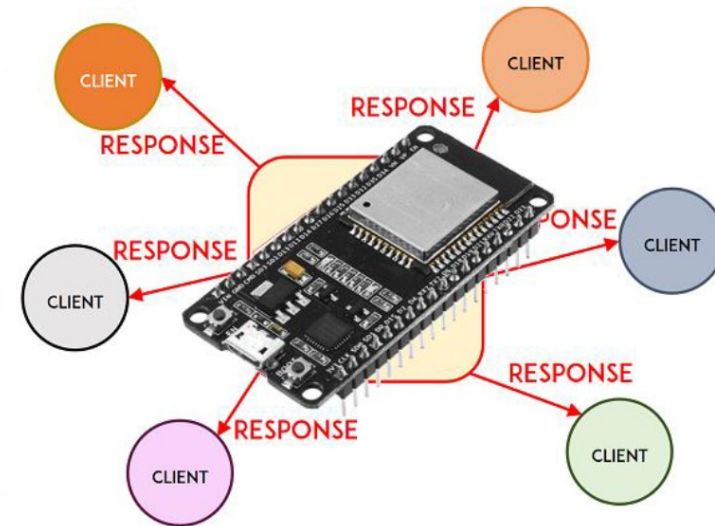
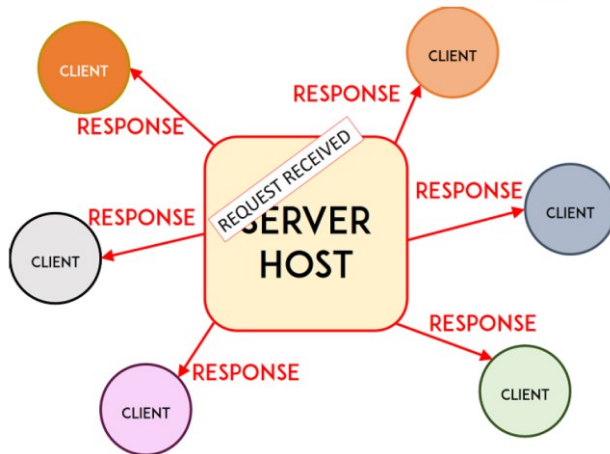
3.3 Code Explanation

3.1 Building ESP32 as a Local Web Server

1. Web Server

How the **ESP32** uses this:

- a) We can run a web server directly on the ESP32.
- b) This allows the ESP32 to “host” a web page that you can access from your phone or computer to control outputs or view sensor data.



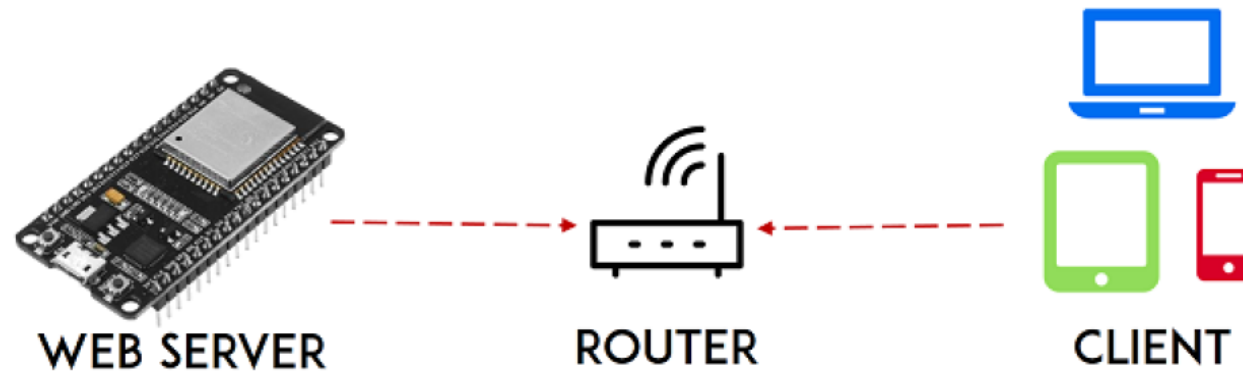
3.1 Building ESP32 as a Local Web Server

2. ESP32 as a Web Server

Let us take a look at a practical example with an IoT device called ESP32 that acts as a web server in the local network.

1) The Local Network:

- a) The ESP32 connects to your Wi-Fi router.
- b) Your computer or smartphone connects to **the same router/access point**.
- c) As a result, all devices are on the **same local network**.

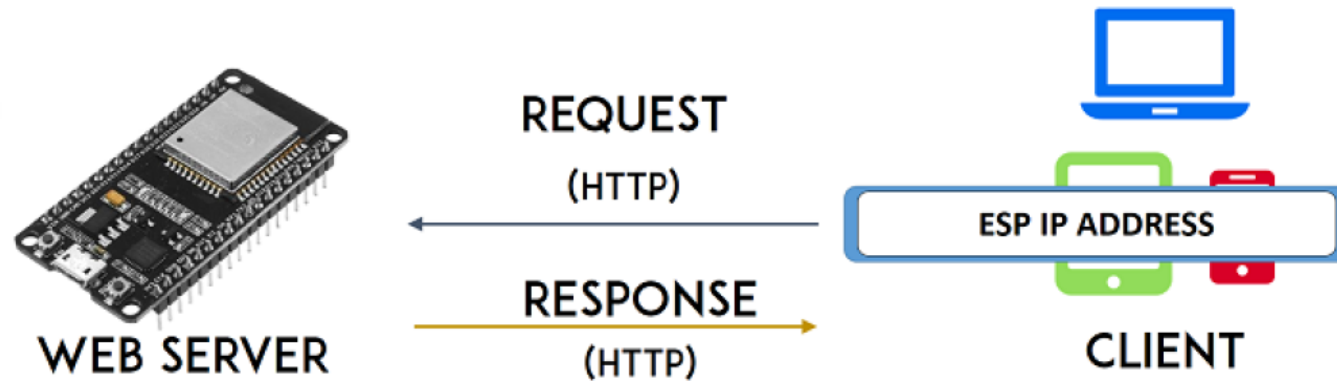


3.1 Building ESP32 as a Local Web Server

2. ESP32 as a Web Server

2) The Interaction:

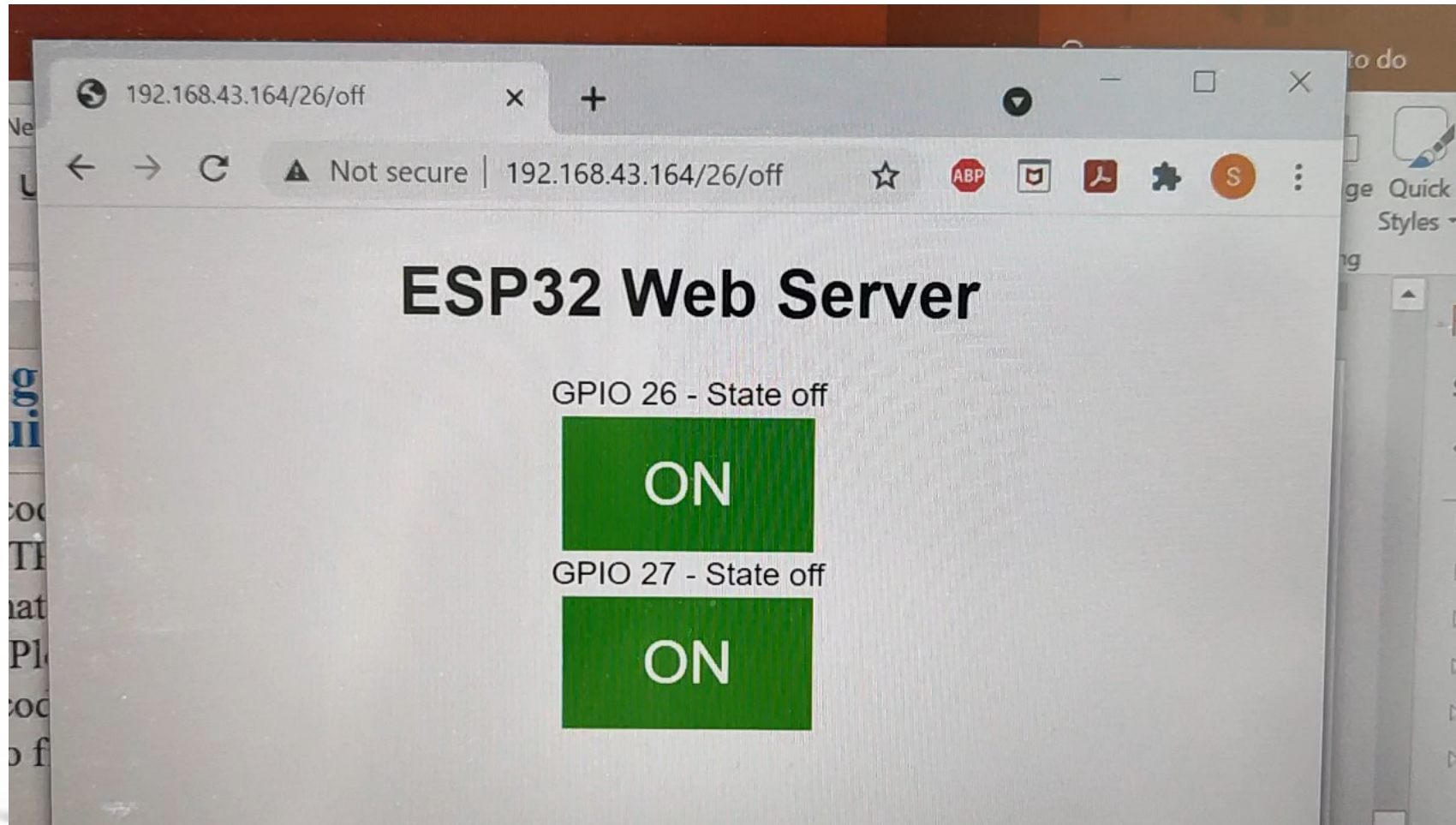
- You type the ESP32's unique IP address into your browser on a client computer.
- Your browser sends an **HTTP Request** directly to the ESP32.
- The ESP32 processes the request and sends back an **HTTP Response**.
- As a result, your browser displays the web page, sensor data, or controls sent by the ESP32.



3.2 ESP32 Web Server to Control Outputs

1. Project Outline

To build a **web server on the ESP32** that hosts a **web page with buttons** to control two LEDs.



3.2 ESP32 Web Server to Control Outputs

1. Project Outline

1) Project Goal:

- To build a **web server on the ESP32** that hosts a **web page with buttons** to control two LEDs.

2) How It Works:

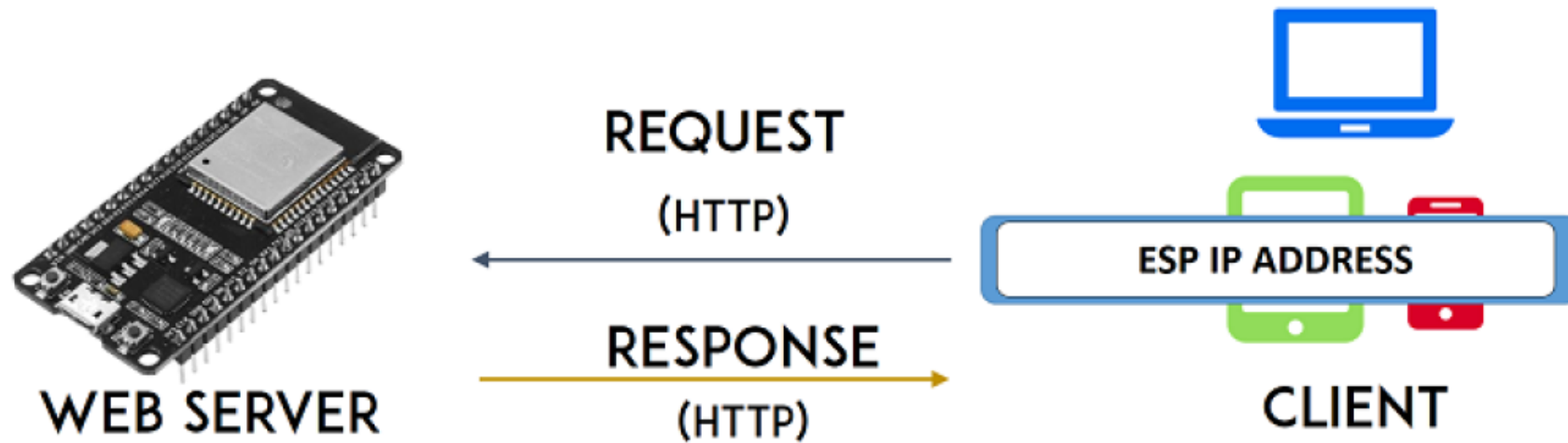
- Connect:** The ESP32 (server) and your computer/phone (client) must be on the same Wi-Fi network.
- Access:** Open a **web browser** on your computer/phone and type in the **ESP32's IP address**.
- Control:** Click the buttons on the **web page** to instantly turn the LEDs (connected to GPIOs 26 & 27) ON or OFF.

3) Beyond this Example:

- This is a foundational project. The LEDs can easily be replaced by relays, motors, or any other actuator you want to control remotely.

3.2 ESP32 Web Server to Control Outputs

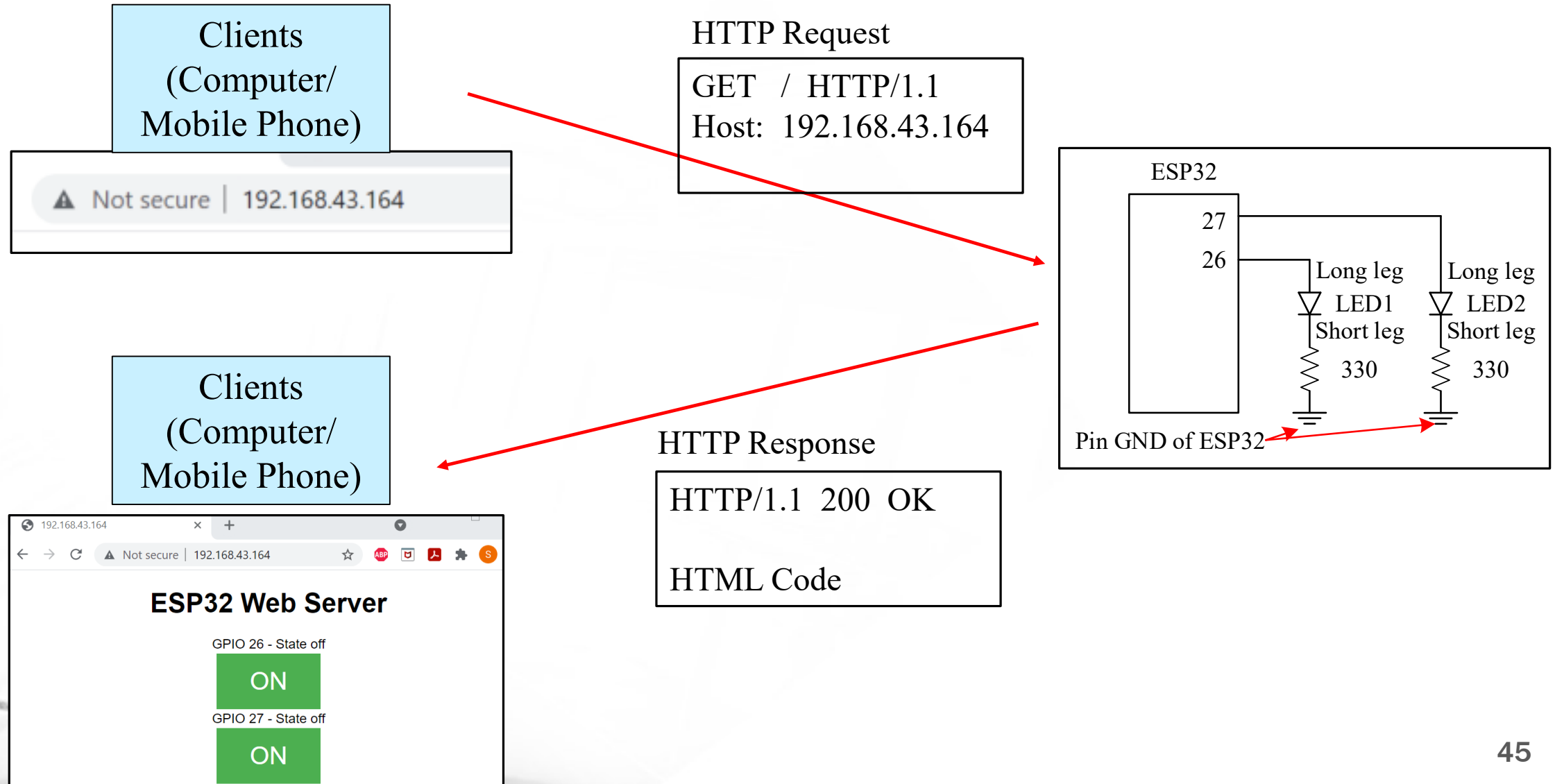
1. Project Outline



3.2 ESP32 Web Server to Control Outputs

1. Project Outline

1) Step 1: On a browser (at Client), the user enters the IP address of the ESP32.

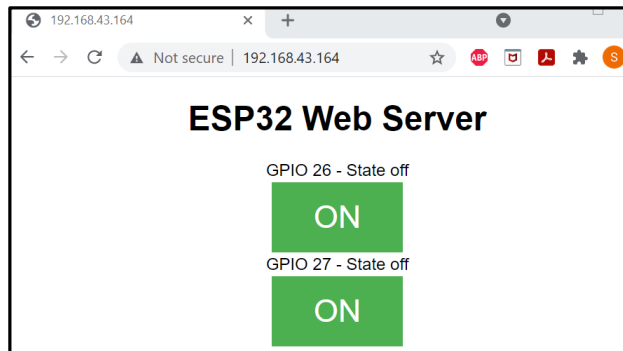


3.2 ESP32 Web Server to Control Outputs

1. Project Outline

2) Step 2: On a browser (at Client), if the user press the ON button of the LED1, the LED1 will be on. The web page will be revised: LED1 state = On and its button is OFF.

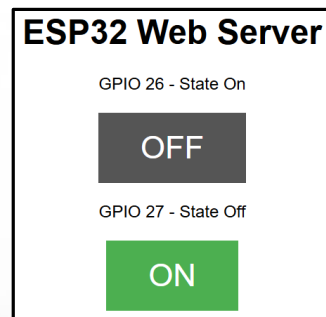
Clients
(Computer/
Mobile Phone)



HTTP Request

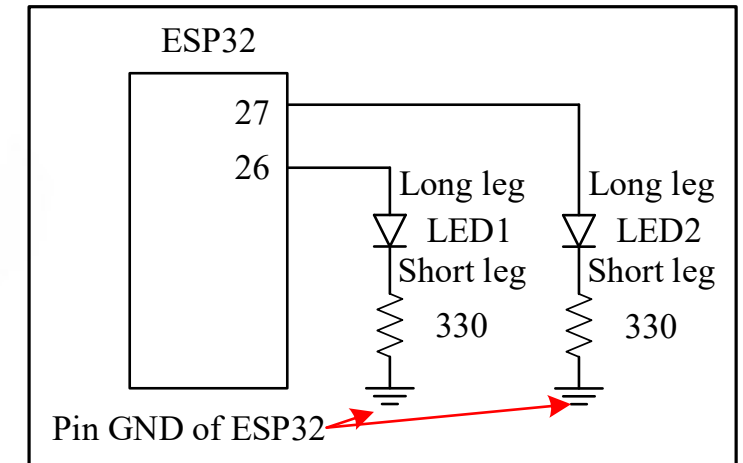
GET /26/on HTTP/1.1
Host: 192.168.43.164

Clients
(Computer/
Mobile Phone)



HTTP Response

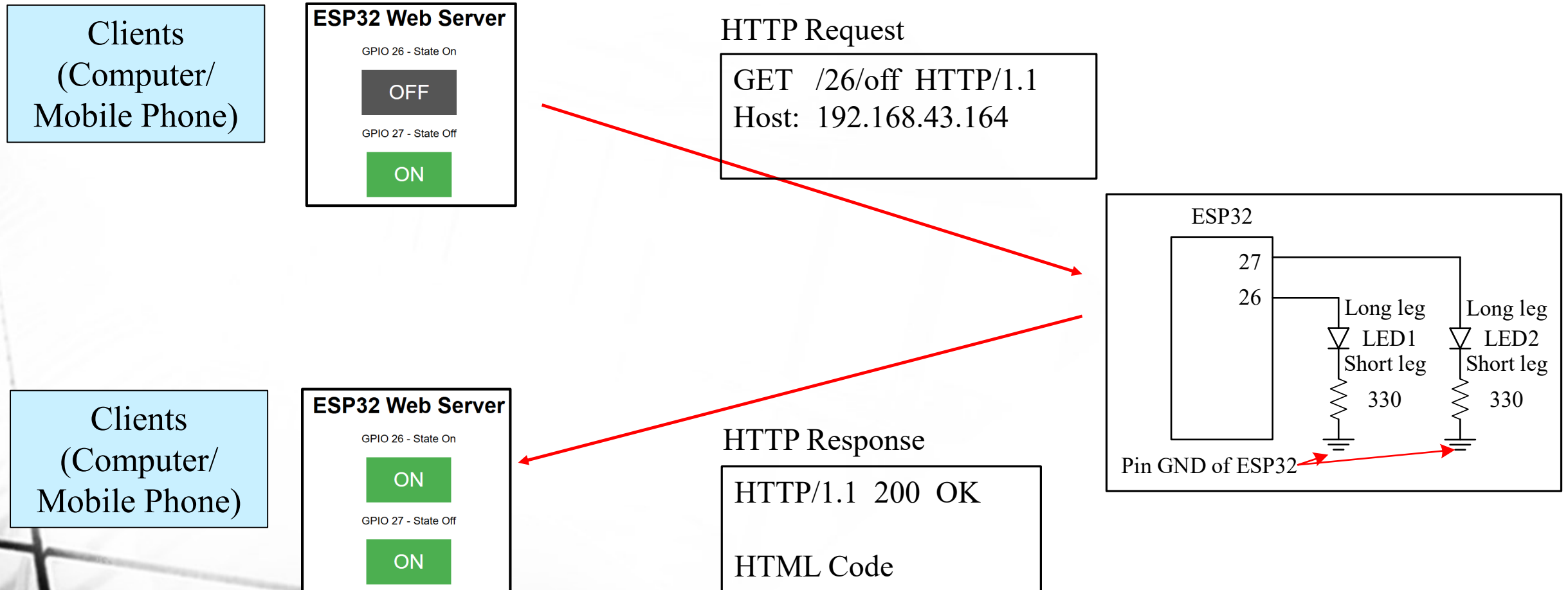
HTTP/1.1 200 OK
HTML Code



3.2 ESP32 Web Server to Control Outputs

1. Project Outline

- 3) Step 3: On a browser (at Client), if the user press the OFF button of the LED1, the LED1 will be off. The web page will be revised: LED1 state = Off and its button is ON.

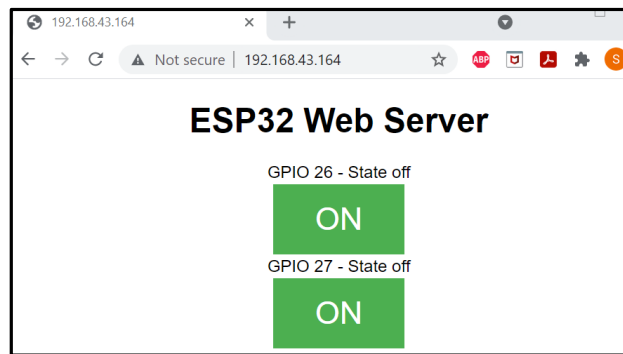


3.2 ESP32 Web Server to Control Outputs

1. Project Outline

- 4) Step 4: On a browser (at Client), if the user press the ON button of the LED2, the LED2 will be on. The web page will be revised: LED2 state = On and its button is OFF.

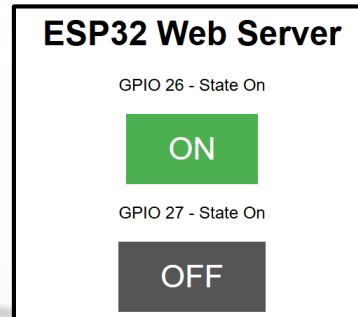
Clients
(Computer/
Mobile Phone)



HTTP Request

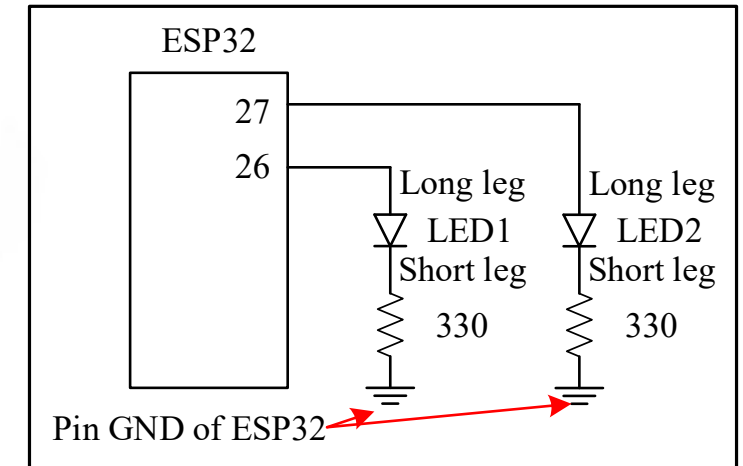
GET /27/on HTTP/1.1
Host: 192.168.43.164

Clients
(Computer/
Mobile Phone)



HTTP Response

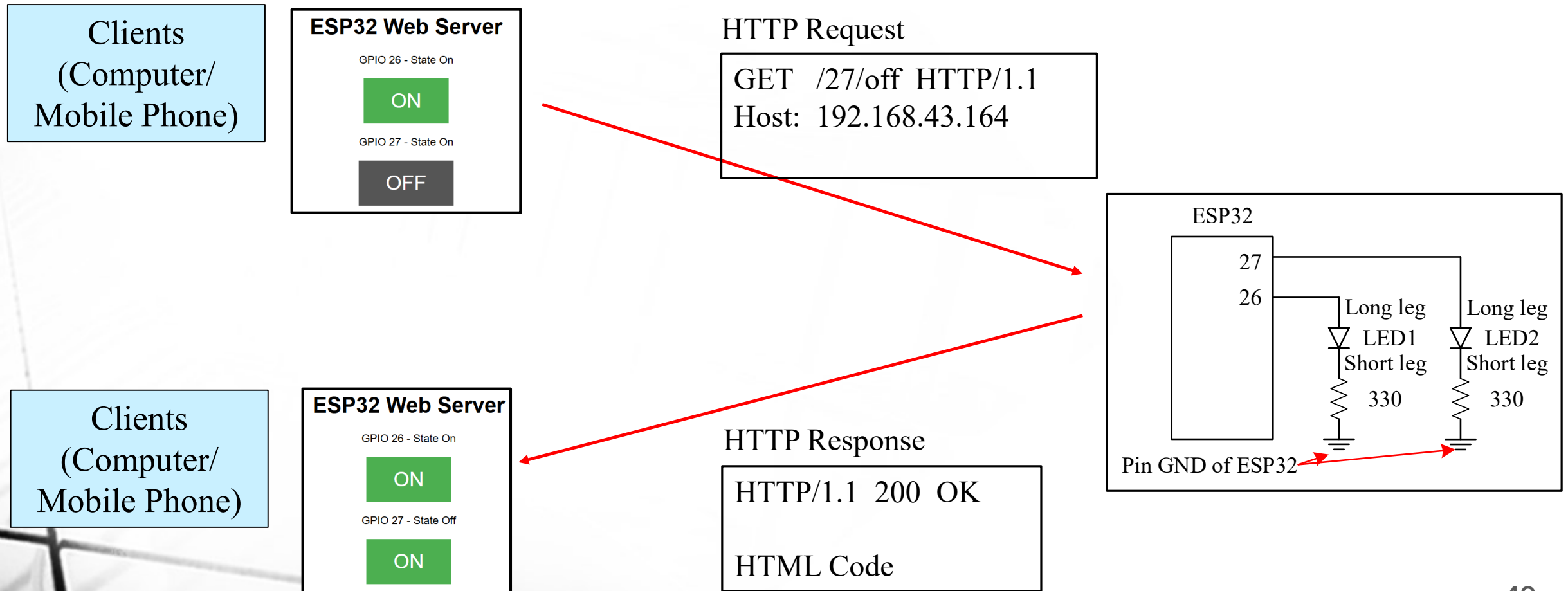
HTTP/1.1 200 OK
HTML Code



3.2 ESP32 Web Server to Control Outputs

1. Project Outline

- 5) Step 5: On a browser (at Client), if the user press the OFF button of the LED2, the LED2 will be off. The web page will be revised: LED2 state = Off and its button is ON.



3.2 ESP32 Web Server to Control Outputs

2. Web Page Design

Our HTML code is similar to what we have designed previously.

1) Case: The LED is OFF and the ON button is active.

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" href="data:,">
  <style>
    html { font-family: Helvetica;
            display: inline-block;
            margin: 0px auto;
            text-align: center;
          }
    .button { background-color: #4CAF50;
              border: none;
              color: white;
              padding: 16px 40px;
              text-decoration: none;
              font-size: 30px;
              margin: 2px;
              cursor: pointer;
            }
    .button2 {background-color: #555555;
              }
  </style>
</head>
```

```
<body>
  <h1>ESP32 Web Server</h1>
  <p>GPIO 26 - State Off</p>
  <p><a href="/26/on"><button class="button">ON</button></a></p>

  <p>GPIO 27 - State Off </p>
  <p><a href="/27/on"><button class="button">ON</button></a></p>
</body>
</html>
```



3.2 ESP32 Web Server to Control Outputs

2. Web Page Design

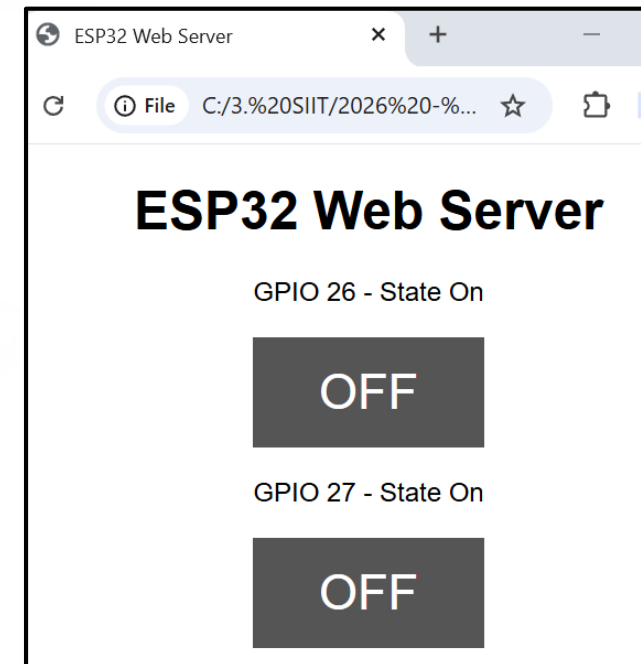
Our HTML code is similar to what we have designed previously.

2) Case: The LED is ON and the OFF button is active.

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" href="data:,">
  <style>
    html { font-family: Helvetica;
            display: inline-block;
            margin: 0px auto;
            text-align: center;
          }
    .button { background-color: #4CAF50;
              border: none;
              color: white;
              padding: 16px 40px;
              text-decoration: none;
              font-size: 30px;
              margin: 2px;
              cursor: pointer;
            }
    .button2 {background-color: #555555;
              }
  </style>
</head>
```

```
<body>
  <h1>ESP32 Web Server</h1>
  <p>GPIO 26 - State On</p>
  <p><a href="/26/on"><button class="button button2">OFF</button></a></p>

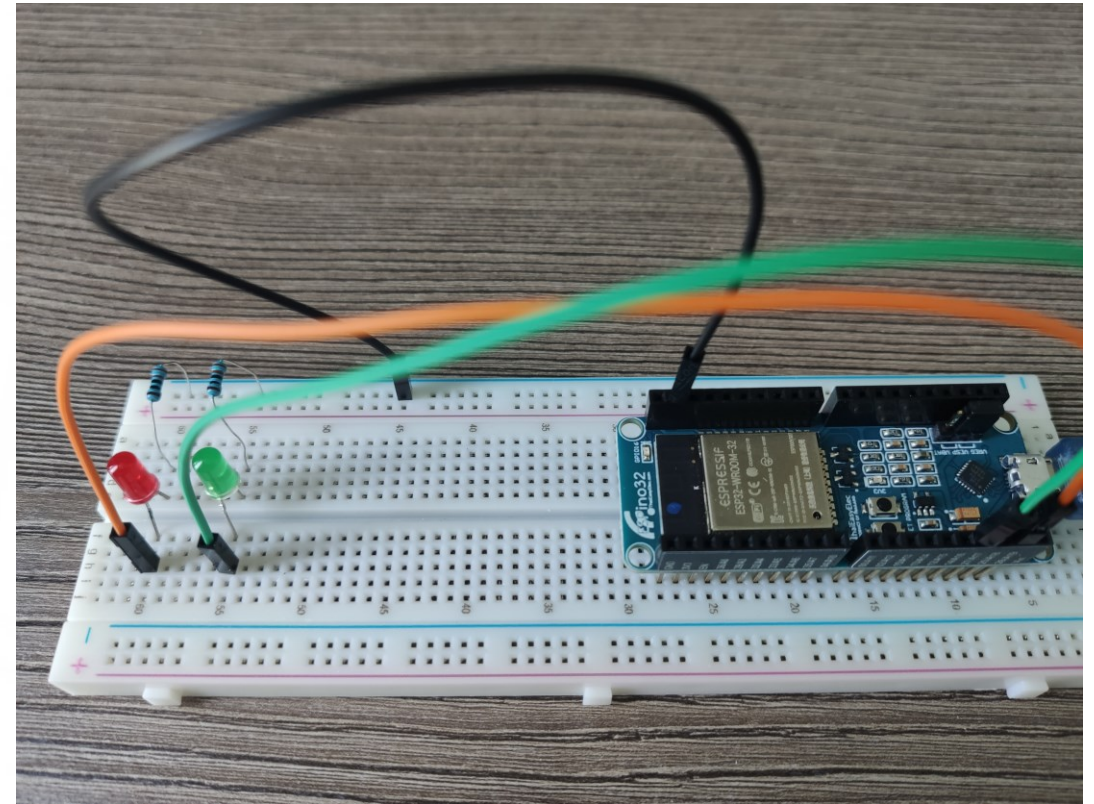
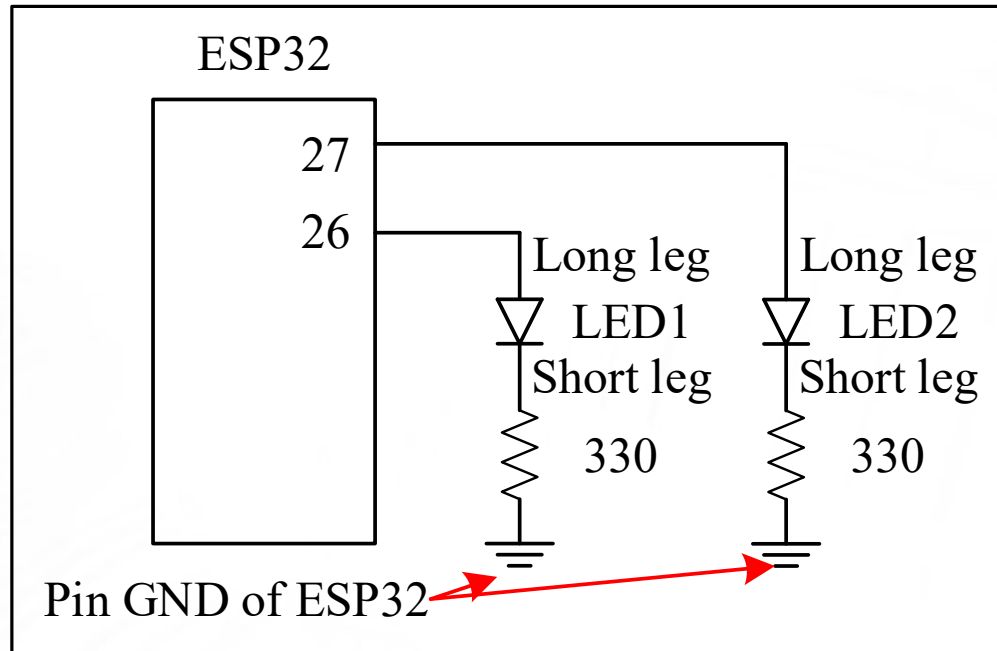
  <p>GPIO 27 - State On </p>
  <p><a href="/27/on"><button class="button button2">OFF</button></a></p>
</body>
</html>
```



3.2 ESP32 Web Server to Control Outputs

3. Schematic

We connect two LEDs to ESP32 as shown in the following schematic diagram – with one LED connected to GPIO 26, and another to GPIO 27.



3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

The Arduino code “**WebServer_ControlOutputs.ino**” will ask ESP32 to send a HTTP response message when it receives the HTTP request from a client (web browser). In side this message, there is the HTML body that we have written in the previous section. The code explanation will be provided later. Please follow the following steps.

- 1) Write this code in your Arduino IDE.
- 2) **You have to fill in your SSID and Password of your access point.**

```
4 // Replace with your network credentials
5 const char* ssid = "Your SSID";
6 const char* password = "Your Password";
```

- 3) Upload the code to your ESP32.

3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

```
1 // Load Wi-Fi library
2 #include <WiFi.h>
3
4 // Replace with your network credentials
5 const char* ssid = "Your SSID";
6 const char* password = "Your Password";
7
8 // Set web server port number to 80
9 WiFiServer server(80);
10
11 // Variable to store the HTTP request
12 String header;
13
14 // Auxiliar variables to store the current output state
15 String output26State = "off";
16 String output27State = "off";
17
18 // Assign output variables to GPIO pins
19 const int output26 = 26;
20 const int output27 = 27;
21
22 // Current time
23 unsigned long currentTime = millis();
24 // Previous time
25 unsigned long previousTime = 0;
26 // Define timeout time in milliseconds (example: 2000ms = 2s)
27 const long timeoutTime = 2000;
28
```


3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

```
29 void setup() {  
30     Serial.begin(115200);  
31     // Initialize the output variables as outputs  
32     pinMode(output26, OUTPUT);  
33     pinMode(output27, OUTPUT);  
34     // Set outputs to LOW  
35     digitalWrite(output26, LOW);  
36     digitalWrite(output27, LOW);  
37  
38     // Connect to Wi-Fi network with SSID and password  
39     Serial.print("Connecting to ");  
40     Serial.println(ssid);  
41     WiFi.mode(WIFI_STA);  
42     WiFi.begin(ssid, password);  
43     while (WiFi.status() != WL_CONNECTED) {  
44         delay(500);  
45         Serial.print(".");  
46     }  
47     // Print local IP address and start web server  
48     Serial.println("");  
49     Serial.println("WiFi connected.");  
50     Serial.println("IP address: ");  
51     Serial.println(WiFi.localIP());  
52     server.begin();  
53 }  
54
```

3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

```
55 void loop(){
56   WiFiClient client = server.available();   // Listen for incoming clients
57
58   if (client) {                             // If a new client connects,
59     currentTime = millis();
60     previousTime = currentTime;
61     Serial.println("New Client.");           // print a message out in the serial port
62     String currentLine = "";                 // make a String to hold incoming data from the client
63     while (client.connected() && currentTime - previousTime <= timeoutTime) { // loop while the client's connected
64       currentTime = millis();
65       if (client.available()) {               // if there's bytes to read from the client,
66         char c = client.read();               // read a byte, then
67         Serial.write(c);                      // print it out the serial monitor
68         header += c;
69         if (c == '\n') {                      // if the byte is a newline character
70           // if the current line is blank, you got two newline characters in a row.
71           // that's the end of the client HTTP request, so send a response:
72           if (currentLine.length() == 0) {
73             // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
74             // and a content-type so the client knows what's coming, then a blank line:
75             client.println("HTTP/1.1 200 OK");
76             client.println("Content-type:text/html");
77             client.println("Connection: close");
78             client.println();
79
```


3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

```
80 // turns the GPIOs on and off
81 if (header.indexOf("GET /26/on") >= 0) {
82     Serial.println("GPIO 26 on");
83     output26State = "on";
84     digitalWrite(output26, HIGH);
85 } else if (header.indexOf("GET /26/off") >= 0) {
86     Serial.println("GPIO 26 off");
87     output26State = "off";
88     digitalWrite(output26, LOW);
89 } else if (header.indexOf("GET /27/on") >= 0) {
90     Serial.println("GPIO 27 on");
91     output27State = "on";
92     digitalWrite(output27, HIGH);
93 } else if (header.indexOf("GET /27/off") >= 0) {
94     Serial.println("GPIO 27 off");
95     output27State = "off";
96     digitalWrite(output27, LOW);
97 }
98
```

- To turn the LED on or off depending on the HTTP Request.
- Set the state of each LED.

3.2 ESP32 Web Server to Control Outputs

4. Arduino Code

HTML File

```
99 // Display the HTML web page
100 client.println("<!DOCTYPE html><html>");
101 client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
102 client.println("<link rel=\"icon\" href=\"data:,\>");
103 // CSS to style the on/off buttons
104 // Feel free to change the background-color and font-size attributes to fit your preferences
105 client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align: center;}");
106 client.println(".button { background-color: #4CAF50; border: none; color: white; padding: 16px 40px;");
107 client.println("text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}");
108 client.println(".button2 {background-color: #555555;}</style></head>");
109
110 // Web Page Heading
111 client.println("<body><h1>ESP32 Web Server</h1>");
112
113 // Display current state, and ON/OFF buttons for GPIO 26
114 client.println("<p>GPIO 26 - State " + output26State + "</p>");
115 // If the output26State is off, it displays the ON button
116 if (output26State=="off") {
117     client.println("<p><a href=\"/26/on\"><button class=\"button\">ON</button></a></p>");
118 } else {
119     client.println("<p><a href=\"/26/off\"><button class=\"button button2\">OFF</button></a></p>");
120 }
121
```

3.2 ESP32 Web Server to Control Outputs

HTML File

4. Arduino Code

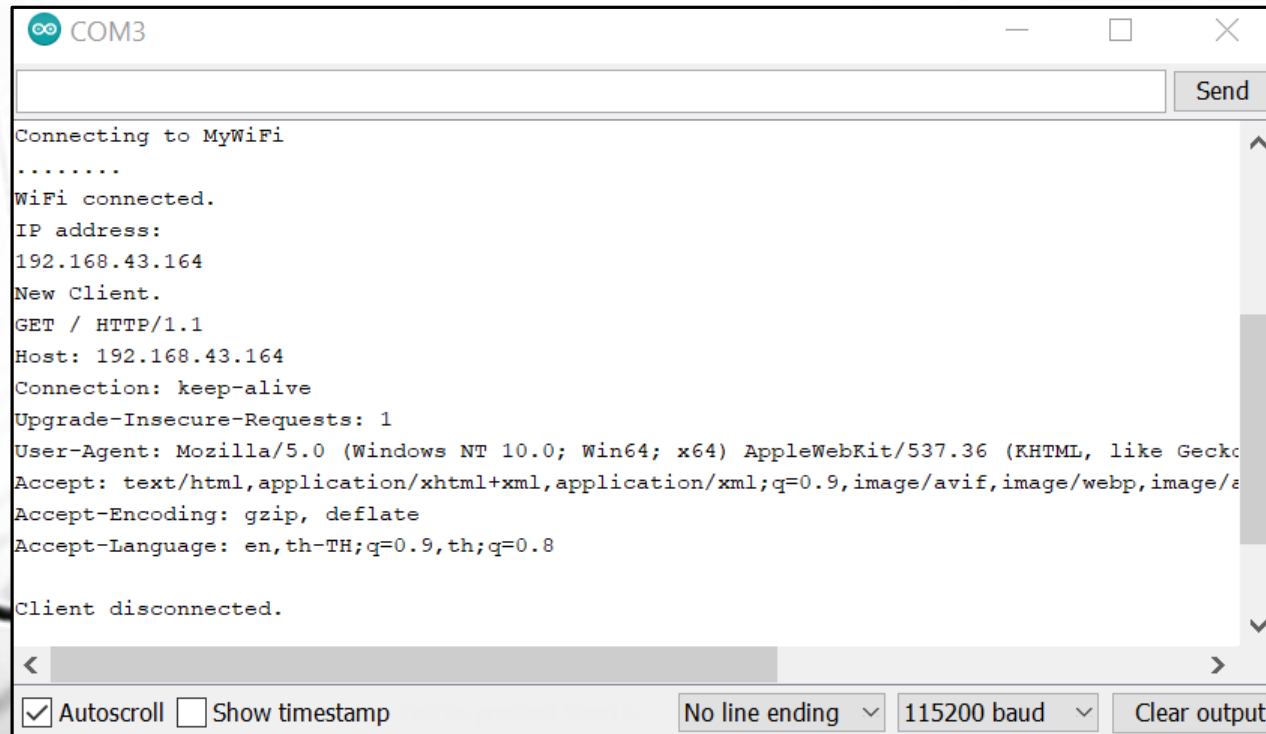
```
122 // Display current state, and ON/OFF buttons for GPIO 27
123 client.println("<p>GPIO 27 - State " + output27State + "</p>");
124 // If the output27State is off, it displays the ON button
125 if (output27State=="off") {
126     client.println("<p><a href=\"/27/on\"><button class=\"button\">ON</button></a></p>");
127 } else {
128     client.println("<p><a href=\"/27/off\"><button class=\"button button2\">OFF</button></a></p>");
129 }
130 client.println("</body></html>");
131
132 // The HTTP response ends with another blank line
133 client.println();
134 // Break out of the while loop
135 break;
136 } else { // if you got a newline, then clear currentLine
137     currentLine = "";
138 }
139 } else if (c != '\r') { // if you got anything else but a carriage return character,
140     currentLine += c; // add it to the end of the currentLine
141 }
142 }
143 }
144 // Clear the header variable
145 header = "";
146 // Close the connection
147 client.stop();
148 Serial.println("Client disconnected.");
149 Serial.println("");
150 }
151 }
```

3.2 ESP32 Web Server to Control Outputs

5. Demonstration

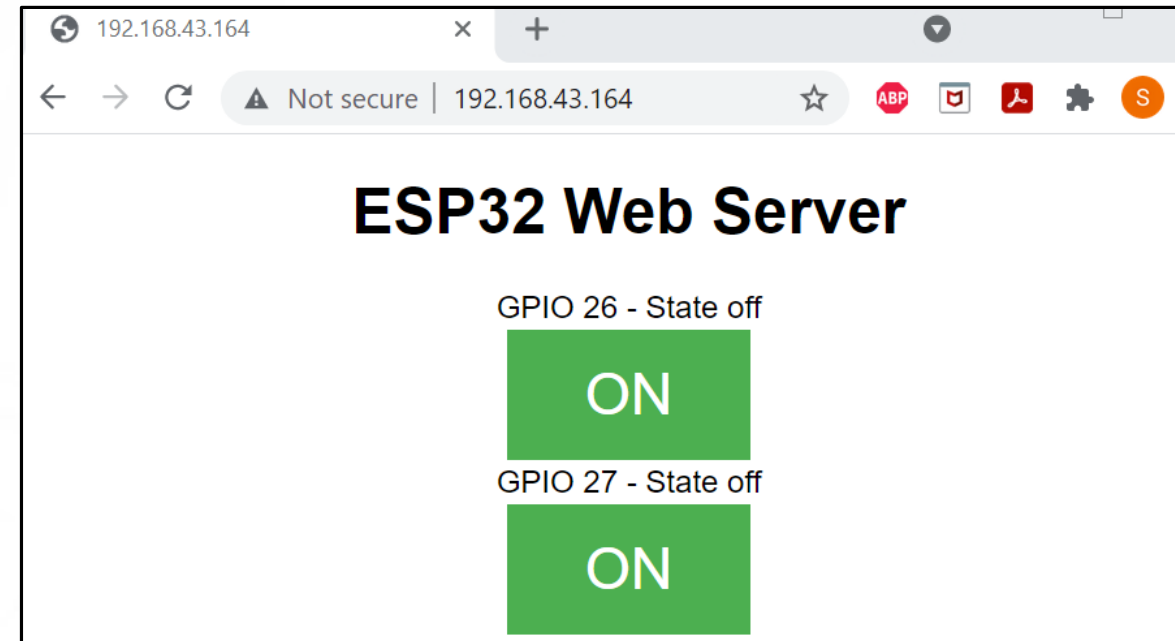
To see the result, please follow these steps.

- 1) When finishing uploading the code to ESP32, the **serial monitor will show the IP address of ESP32** which right now is working as a web server.
- 2) **Connect your computer/phone to the same access point (that ESP32 is connecting to).** On a web browser, type the IP address (of ESP32) on the URL and press the Enter. You will see this web page. Clicking the buttons will turn the LEDs on/off.



```
COM3
Connecting to MyWiFi
.....
WiFi connected.
IP address:
192.168.43.164
New Client.
GET / HTTP/1.1
Host: 192.168.43.164
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/e
Accept-Encoding: gzip, deflate
Accept-Language: en,th-TH;q=0.9,th;q=0.8

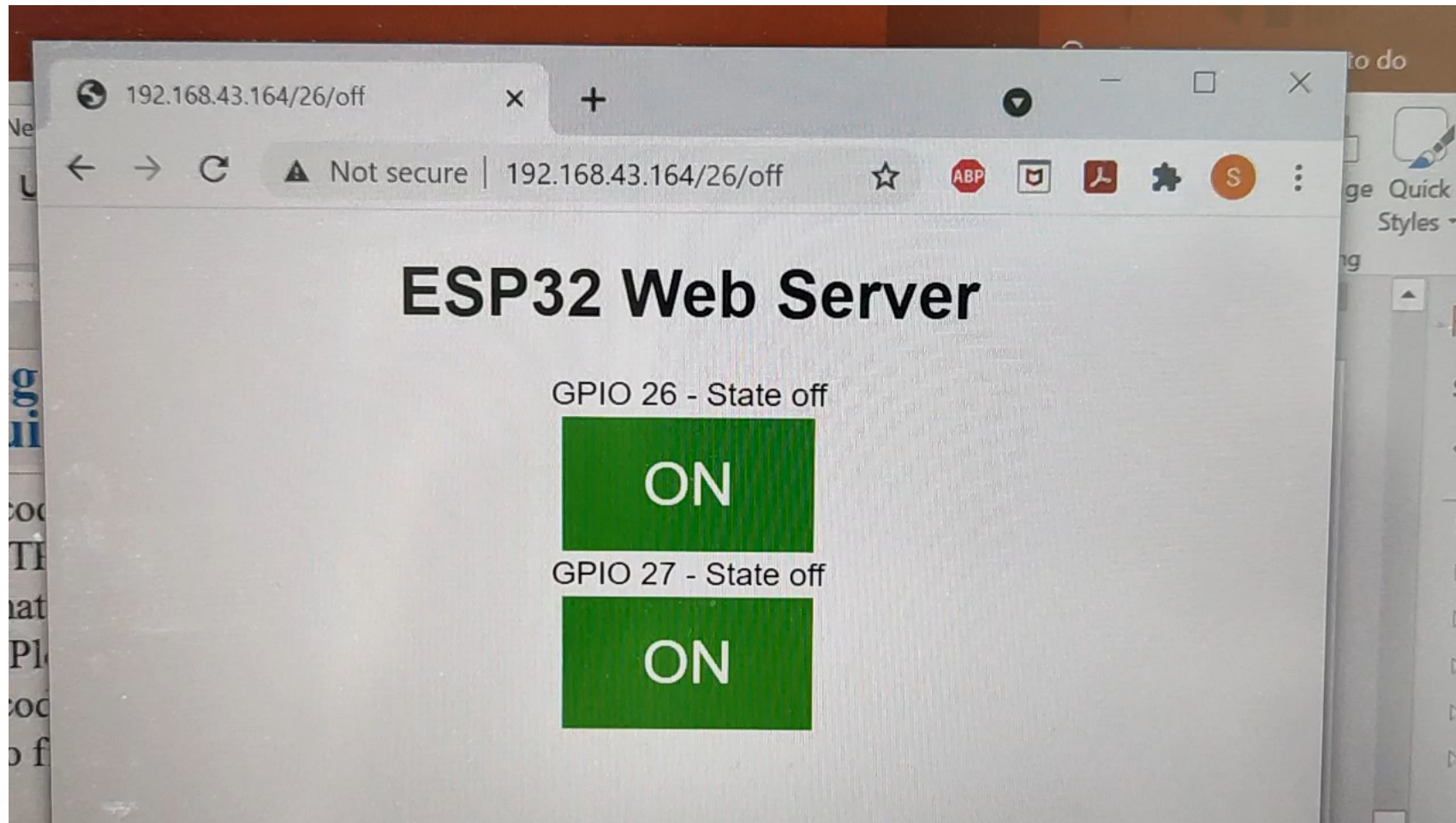
Client disconnected.
```



3.2 ESP32 Web Server to Control Outputs

5. Demonstration

The demonstration video is shown below.



3.3 Code Explanation

Mainly the code consists of the following parts.

- 1) Set ESP32 as a webserver.

```
8 // Set web server port number to 80
9 WiFiServer server(80);
```

- 2) Connect to the access point.

```
41 WiFi.mode(WIFI_STA);
42 WiFi.begin(ssid, password);
43 while (WiFi.status() != WL_CONNECTED) {
```


3.3 Code Explanation

3) Listen to a client and receive the HTTP request (stored it in the variable named *header*)

```
58  if (client) {
```

```
65      if (client.available()) {  
66          char c = client.read();  
67          Serial.write(c);  
68          header += c;
```

Note:

- At the first time when the user enters the IP address of ESP32 at a browser of the client, the client will send a HTTP request: “GET / /HTTP1.1” to the ESP32.
- Later, after the client receive the HTTP response including the HTML source code, our browser will display the webpage.
 - If we click the **ON button of the pin 26**, there will be the text “GET /26/on” in the HTTP request; turn the LED at pin 26 on.
 - If we click the **OFF button of the pin 26**, there will be the text “GET /26/off” in the HTTP request; turn the LED at pin 26 off.
 - If we click the **ON button of the pin 27**, there will be the text “GET /27/on” in the HTTP request; turn the LED at pin 27 on.
 - If we click the **OFF button of the pin 27**, there will be the text “GET /27/off” in the HTTP request; turn the LED at pin 27 off.

3.3 Code Explanation

- 4) Check whether the HTTP request stores “GET /26/on”, “GET /26/off”, “GET /27/on”, or “GET /27/off”. Then turn on/off the LED correspondingly.

```
80 // turns the GPIOs on and off
81 if (header.indexOf("GET /26/on") >= 0) {
82     Serial.println("GPIO 26 on");
83     output26State = "on";
84     digitalWrite(output26, HIGH);
85 } else if (header.indexOf("GET /26/off") >= 0) {
86     Serial.println("GPIO 26 off");
87     output26State = "off";
88     digitalWrite(output26, LOW);
89 } else if (header.indexOf("GET /27/on") >= 0) {
90     Serial.println("GPIO 27 on");
91     output27State = "on";
92     digitalWrite(output27, HIGH);
93 } else if (header.indexOf("GET /27/off") >= 0) {
94     Serial.println("GPIO 27 off");
95     output27State = "off";
96     digitalWrite(output27, LOW);
97 }
```

The *.indexOf()* function is to check whether we have the specified text in the specified string variable.

The *header.indexOf()* (“GET /26/on”) command will return:

- **>=0** if there is “GET /26/on” in the *header* variable.
- **-1** if there is no “GET /26/on” in the *header* variable.

where here, the *header* variable store the HTTP request message.

3.3 Code Explanation

- 5) Send the HTTP response message (including the HTML body) to the client (Line 99-133).

```
99      // Display the HTML web page
100     client.println("<!DOCTYPE html><html>");
101     client.println("<head><meta name=\"viewport\"");
102     client.println("<link rel=\"icon\" href=\"dat");
103     // CSS to style the on/off buttons
104     // Feel free to change the background-color a
105     client.println("<style>html { font-family: He");
106     client.println("<button { background-color: #
```

- The variable name `client` is used to represent the current client that is connecting to ESP32. It has been defined in Line 56.

```
56  WiFiClient client = server.available();
```

- The *`client.println()`* function will send the message inside the “” to the client. We can notice that a series of texts sent by the *`client.println()`* function is the HTTP response message.

4. ESP32 as a Local Web Server to Monitor Sensor Data

This section consists of

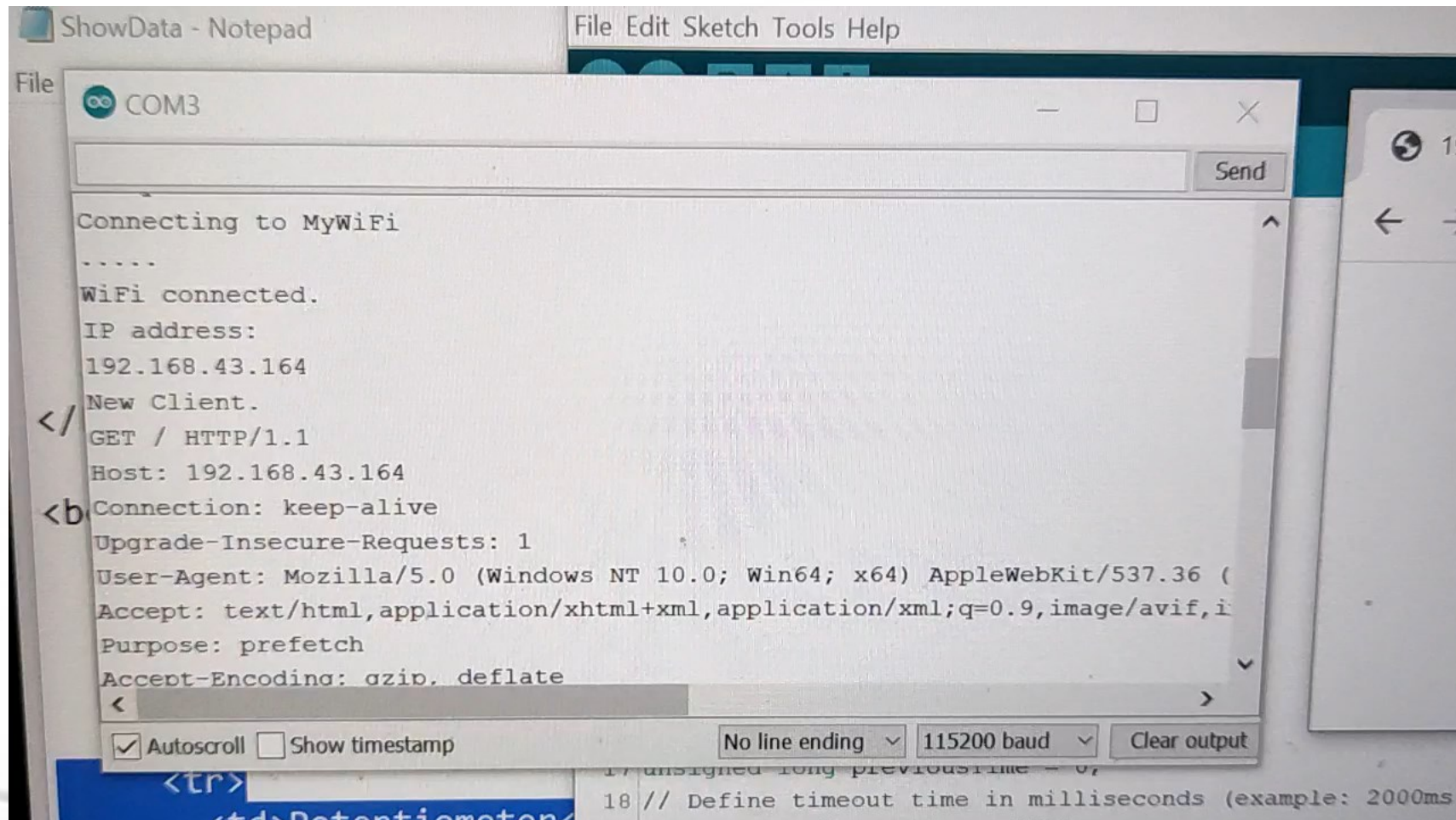
4.1 ESP32 Web Server to Monitor Sensor Data

4.2 Code Explanation

4.1 ESP32 Web Server to Monitor Sensor Data

1. Project Outline

To build a **web server** on the **ESP32** that **displays readings** from a **potentiometer** (connected to pin A0).



The screenshot shows the Arduino IDE interface with a serial monitor window titled 'COM3'. The monitor displays the following text:

```
Connecting to MyWiFi
.....
WiFi connected.
IP address:
192.168.43.164
</ GET / HTTP/1.1
Host: 192.168.43.164
<b Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
Purpose: prefetch
Accept-Encoding: gzip, deflate
<
17 unsigned long previousTime = 0;
18 // Define timeout time in milliseconds (example: 2000ms)
```

The serial monitor window has a 'Send' button and a 'Clear output' button. The background shows the Arduino IDE code editor with a snippet of HTML code:

```
<tr>
<td>Potentiometer</td>
```

4.1 ESP32 Web Server to Monitor Sensor Data

1. Project Outline

1) Project Goal:

- To build a **web server on the ESP32** that **displays readings from a potentiometer** (connected to pin A0).

2) How It Works:

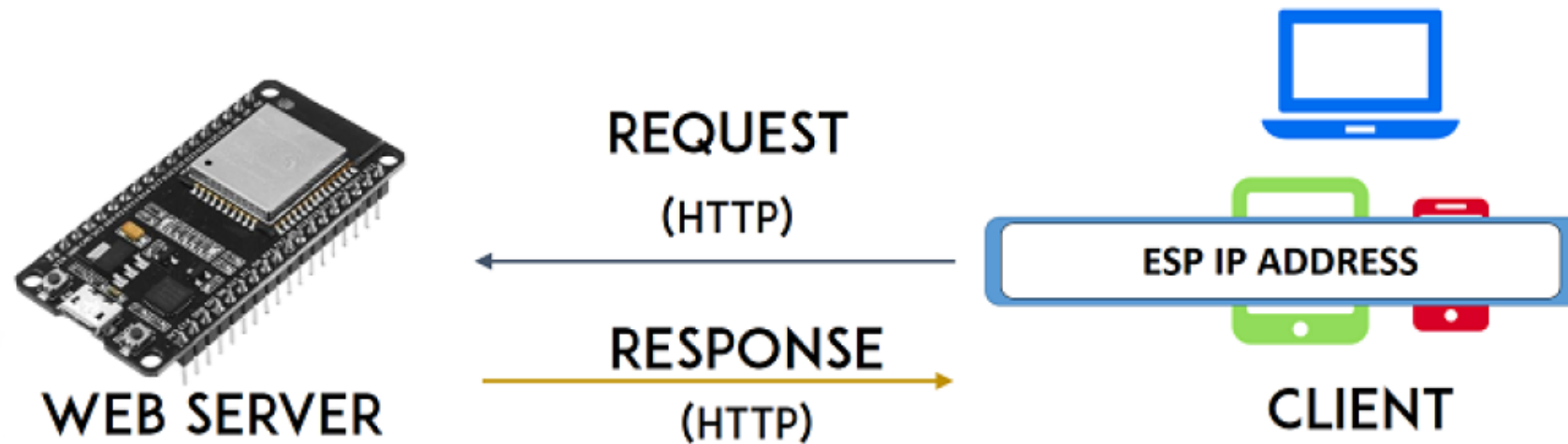
- Connect:** The ESP32 (server) and your computer/phone (client) must be on the same Wi-Fi network.
- Access:** Open a **web browser** and type in the **ESP32's IP address**.
- Monitor:** The web page will display the real-time value from the potentiometer.

3) Beyond this Example:

- This project is a template for monitoring any sensor. You can easily add more sensors (temperature, humidity, etc.) and display all their data on one dashboard.

4.1 ESP32 Web Server to Monitor Sensor Data

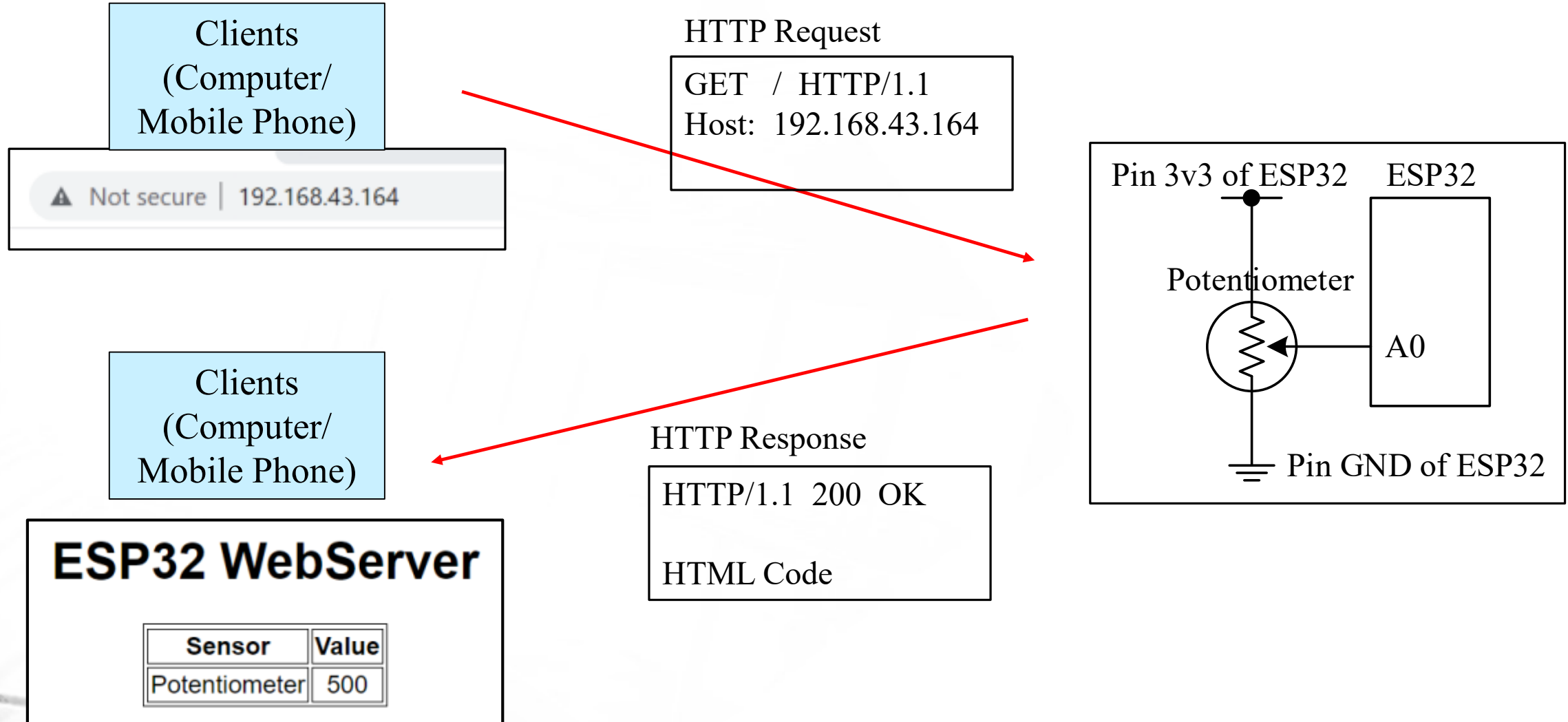
1. Project Outline



4.1 ESP32 Web Server to Monitor Sensor Data

1. Project Outline

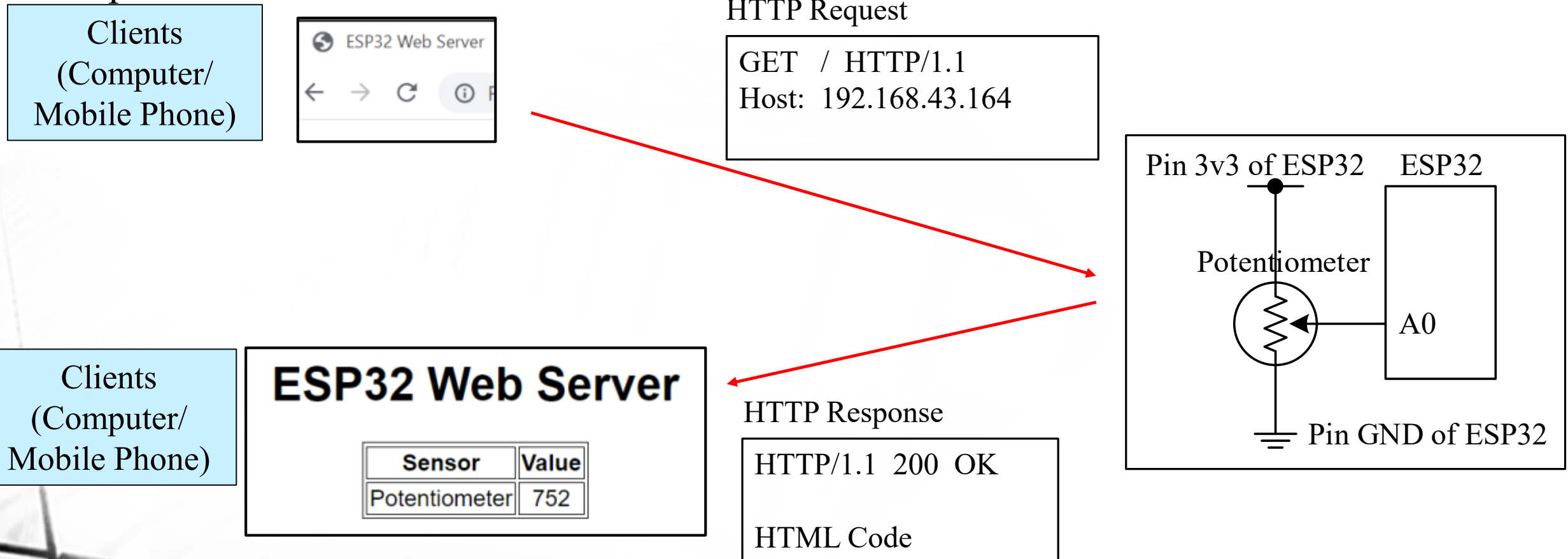
- 1) Step 1: On a browser (at Client), the user enters the IP address of the ESP32.



4.1 ESP32 Web Server to Monitor Sensor Data

1. Project Outline

- 2) Step 2: The user rotate the knob of the potentiometer to change its value. Thereafter, the user presses the refresh button on the browser. The web page will update the potentiometer value.



4.1 ESP32 Web Server to Monitor Sensor Data

2. Web Page Design

In this project, we will **display the sensor data in a table in a web page**. Our HTML code is similar to the previous one except we create a table consisting of this two steps.

(More details can be found from here: https://www.w3schools.com/html/html_tables.asp)

- 1) In the <style> tags, we will define the style of our table as follows, where we set up the table border width to 1 pixel and locate the table at the center of the page.

```
table, th, td {  
    border: 1px solid black;  
    margin-left:auto;  
    margin-right:auto;  
}
```

4.1 ESP32 Web Server to Monitor Sensor Data

2. Web Page Design

- 2) In the <body> tags, we create a table by using the following tags.
- The <table> tag defines an HTML table.
 - The <th> tag defines the table header. By default, the text in <th> elements are bold and centered.
 - The <tr> tag defines a row.
 - The <td> tag defines a cell in a row. By default, the text in <td> elements are regular and left-aligned.

We create the table as shown below.

```
<table>
  <tr>
    <th>Sensor</th><th>Value</th>
  </tr>
  <tr>
    <td>Potentiometer</td><td>500</td>
  </tr>
</table>
```

4.1 ESP32 Web Server to Monitor Sensor Data

2. Web Page Design

Our complete HTML code is shown below.

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 Web Server</title>
  <meta name="viewport" content="width=device-width, initialscale=1">
  <link rel="icon" href="data:,">
  <style>
    html { font-family: Helvetica;
           display: inline-block;
           margin:0px auto;
           text-align: center;
         }
    table, th, td {
      border: 1px solid black;
      margin-left:auto;
      margin-right:auto;
    }
  </style>
</head>
```

4.1 ESP32 Web Server to Monitor Sensor Data

2. Web Page Design

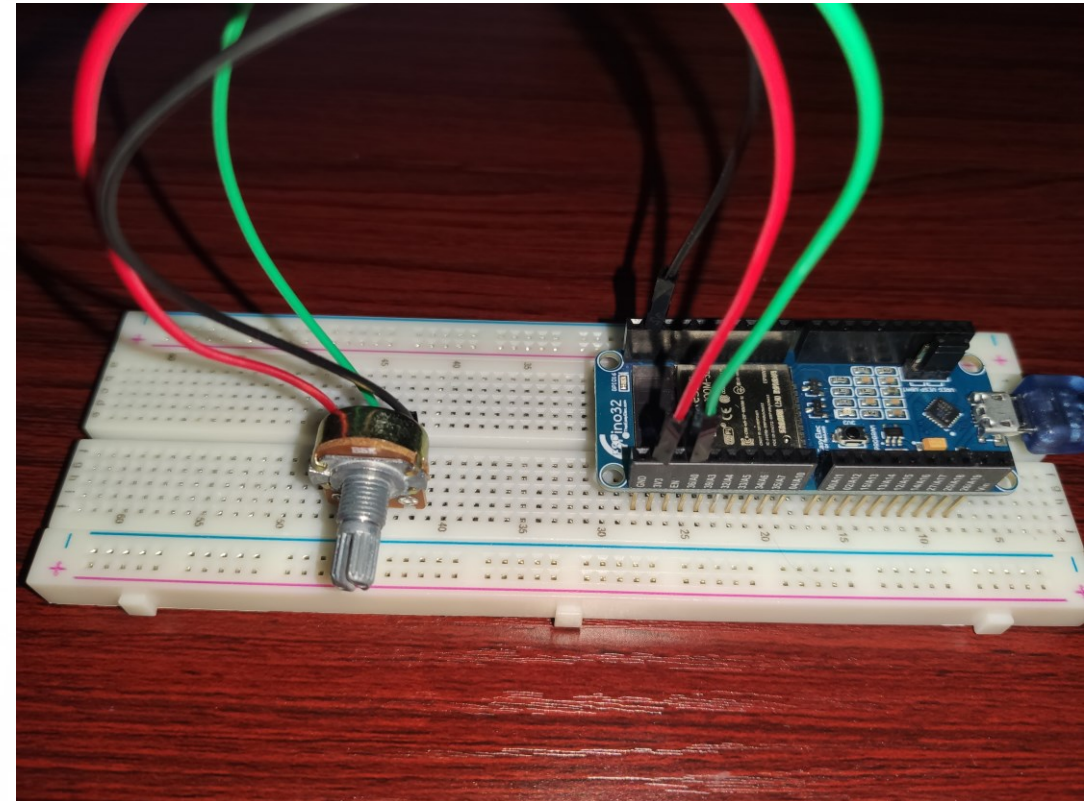
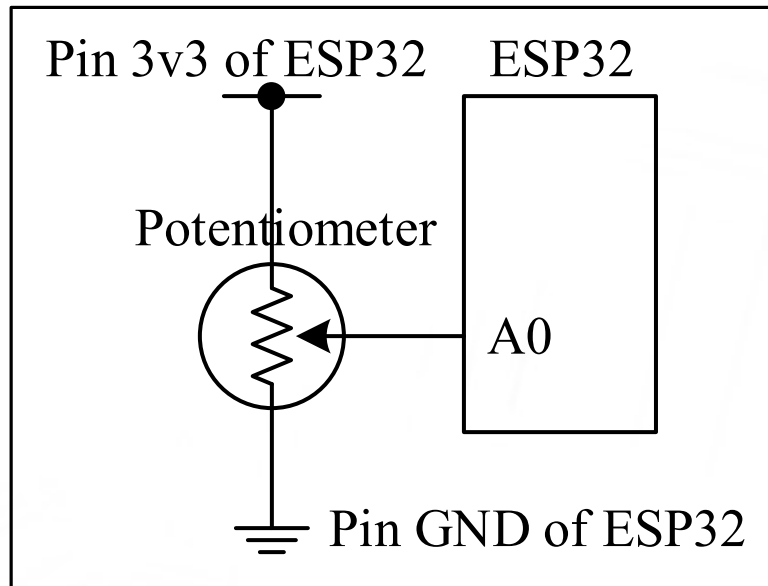
```
<body>
  <h1>ESP32 WebServer</h1>
  <table>
    <tr>
      <th>Sensor</th><th>Value</th>
    </tr>
    <tr>
      <td>Potentiometer</td><td>500</td>
    </tr>
  </table>
</body>
</html>
```



4.1 ESP32 Web Server to Monitor Sensor Data

3. Schematic

We connect a potentiometer to the pin A0 of ESP32 as shown in the following schematic diagram.



4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

The Arduino code “**WebServer_MonitorSensor.ino**” will ask ESP32 to send a HTTP response message when it receives the HTTP request from a client (web browser). In side this message, there is the **HTML body (including the potentiometer value)** that we have written in the previous section. The code explanation will be provided later. Please follow the following steps.

- 1) Write this code in your Arduino IDE.
- 2) You have to fill in your SSID and Password of your access point.

```
4 // Replace with your network credentials
5 const char* ssid = "Your SSID";
6 const char* password = "Your Password";
```

- 3) Upload the code to your ESP32.

4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

```
1 // Load Wi-Fi library
2 #include <WiFi.h>
3
4 // Replace with your network credentials
5 const char* ssid = "MyWiFi";
6 const char* password = "Moon15Star18";
7
8 // Set web server port number to 80
9 WiFiServer server(80);
10
11 // Variable to store the HTTP request
12 String header;
13
14 // Current time
15 unsigned long currentTime = millis();
16 // Previous time
17 unsigned long previousTime = 0;
18 // Define timeout time in milliseconds (example: 2000ms = 2s)
19 const long timeoutTime = 2000;
20
```

4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

```
21 void setup() {  
22     Serial.begin(115200);  
23     // Connect to Wi-Fi network with SSID and password  
24     Serial.print("Connecting to ");  
25     Serial.println(ssid);  
26     WiFi.mode(WIFI_STA);  
27     WiFi.begin(ssid, password);  
28     while (WiFi.status() != WL_CONNECTED) {  
29         delay(500);  
30         Serial.print(".");  
31     }  
32     // Print local IP address and start web server  
33     Serial.println("");  
34     Serial.println("WiFi connected.");  
35     Serial.println("IP address: ");  
36     Serial.println(WiFi.localIP());  
37     server.begin();  
38 }  
39
```

4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

```
40 void loop(){
41   WiFiClient client = server.available();   // Listen for incoming clients
42
43   if (client) {                             // If a new client connects,
44     currentTime = millis();
45     previousTime = currentTime;
46     Serial.println("New Client.");          // print a message out in the serial port
47     String currentLine = "";                // make a String to hold incoming data from the client
48     while (client.connected() && currentTime - previousTime <= timeoutTime) { // loop while the client's connected
49       currentTime = millis();
50       if (client.available()) {             // if there's bytes to read from the client,
51         char c = client.read();             // read a byte, then
52         Serial.write(c);                    // print it out the serial monitor
53         header += c;
54         if (c == '\n') {                    // if the byte is a newline character
55           // if the current line is blank, you got two newline characters in a row.
56           // that's the end of the client HTTP request, so send a response:
57           if (currentLine.length() == 0) {
58             // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
59             // and a content-type so the client knows what's coming, then a blank line:
60             client.println("HTTP/1.1 200 OK");
61             client.println("Content-type:text/html");
62             client.println("Connection: close");
63             client.println();
64
```

4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

HTML File

```
65 // Display the HTML web page
66 client.println("<!DOCTYPE html><html>");
67 client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
68 client.println("<link rel=\"icon\" href=\"data:,\">>");
69 // CSS to style the on/off buttons
70 // Feel free to change the background-color and font-size attributes to fit your preferences
71 client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align: center;}");
72 client.println("table, th, td { border: 1px solid black; margin-left:auto; margin-right:auto;}");
73 client.println("</style></head>");
74
75 // Web Page Heading
76 client.println("<body><h1>ESP32 Web Server</h1>");
77
78 // Create a table
79 client.println("<table>");
80 client.println("<tr><th>Sensor</th><th>Value</th></tr>"); // First row
81 client.print("<tr><td>Potentiometer</td><td>"); // Second row
82 client.print(analogRead(A0)); // Read the potentiometer value
83 client.println("</td></tr>"); // Second row
84 client.println("</table>");
85 client.println("</body></html>");
86
87 // The HTTP response ends with another blank line
88 client.println();
89 // Break out of the while loop
90 break;
```

4.1 ESP32 Web Server to Monitor Sensor Data

4. Arduino Code

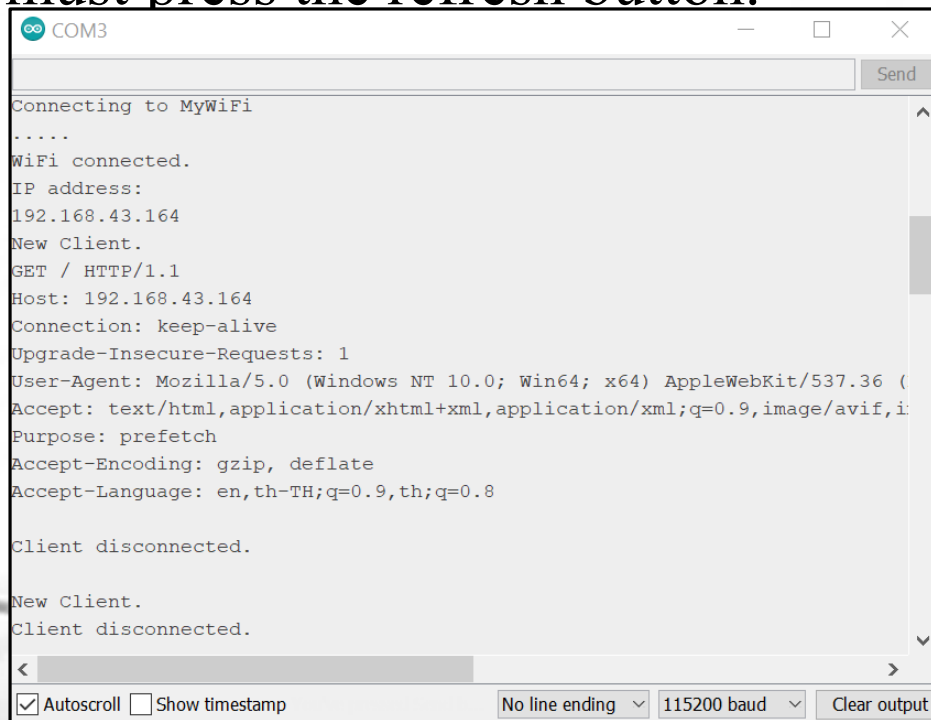
```
91     } else { // if you got a newline, then clear currentLine
92         currentLine = "";
93     }
94     } else if (c != '\r') { // if you got anything else but a carriage return character,
95         currentLine += c;    // add it to the end of the currentLine
96     }
97 }
98 }
99 // Clear the header variable
100 header = "";
101 // Close the connection
102 client.stop();
103 Serial.println("Client disconnected.");
104 Serial.println("");
105 }
106 }
```

4.1 ESP32 Web Server to Monitor Sensor Data

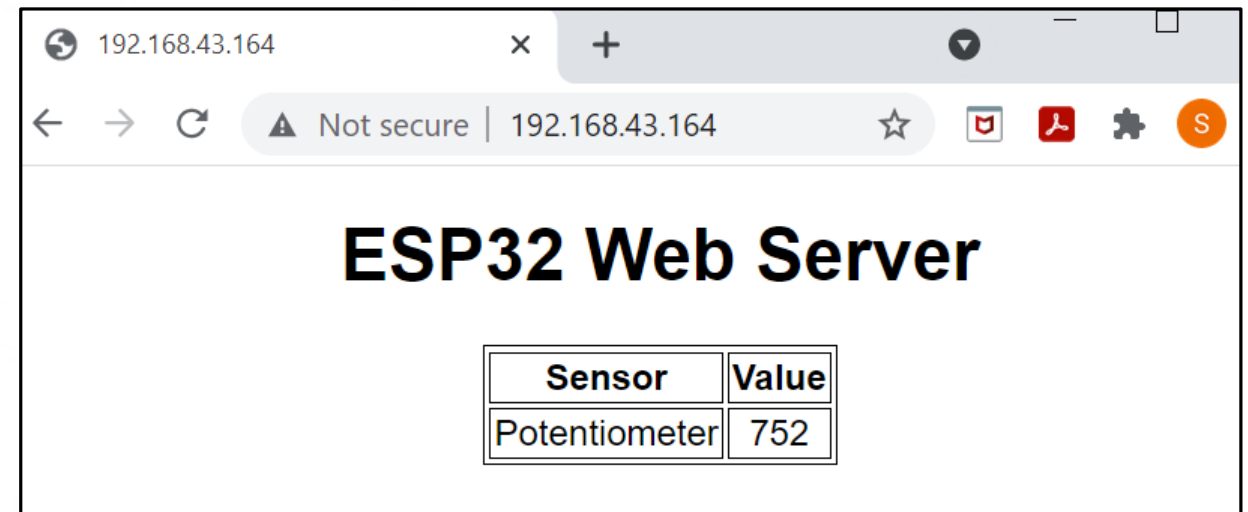
5. Demonstration

To see the result, please follow these steps.

- 1) When finishing uploading the code to ESP32, the serial monitor will show the IP address of ESP32 which right now is working as a web server.
- 2) **Connect your computer/phone to the same access point (that ESP32 is connecting to).** On a web browser, type the IP address (of ESP32) on the URL and press the Enter. You will see this web page show the potentiometer value. To update the value, we must press the refresh button.



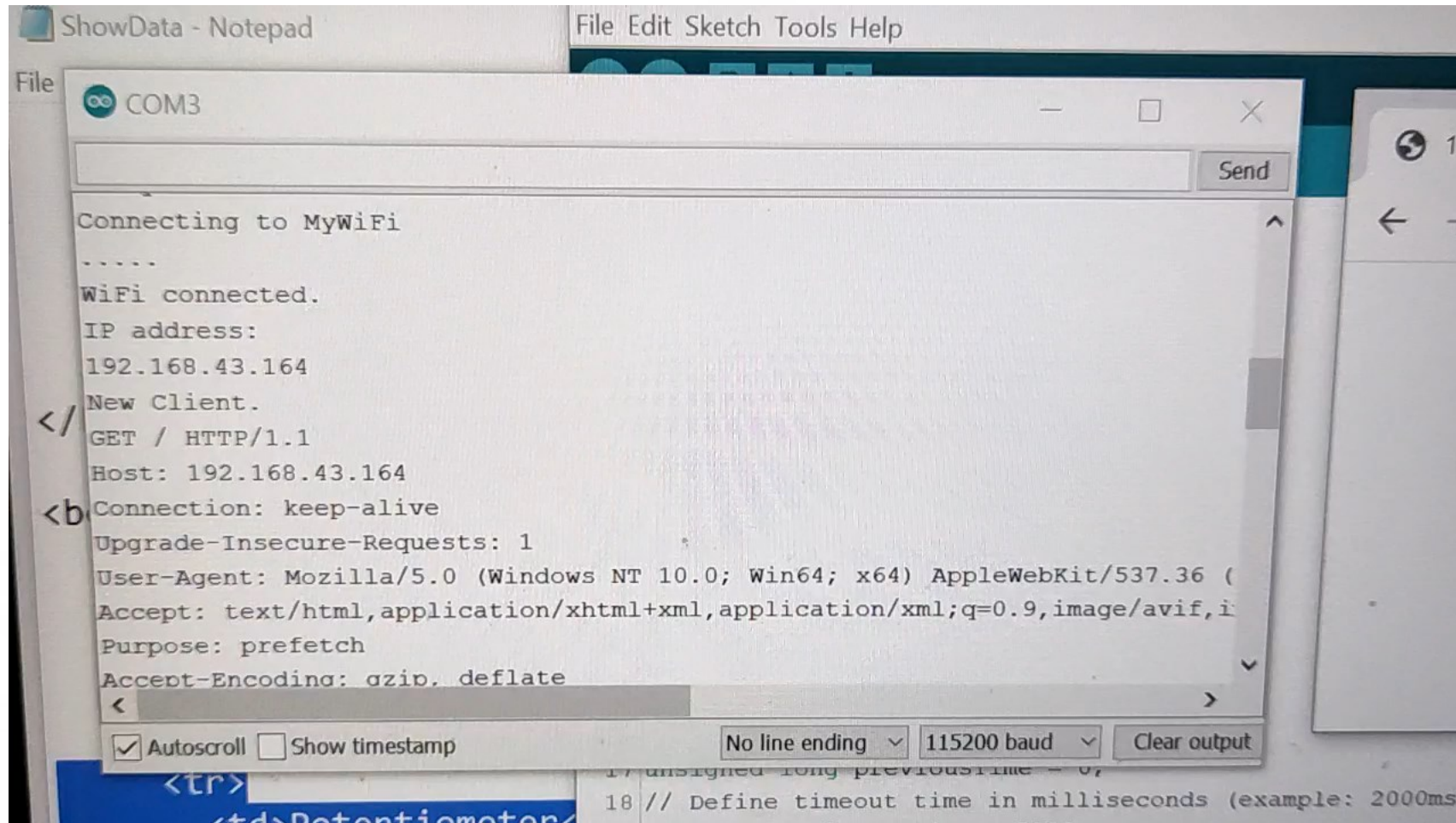
A screenshot of a serial monitor window titled 'COM3'. The window shows the following text: 'Connecting to MyWiFi', 'WiFi connected.', 'IP address: 192.168.43.164', 'New Client.', 'GET / HTTP/1.1', 'Host: 192.168.43.164', 'Connection: keep-alive', 'Upgrade-Insecure-Requests: 1', 'User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (', 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i', 'Purpose: prefetch', 'Accept-Encoding: gzip, deflate', 'Accept-Language: en,th-TH;q=0.9,th;q=0.8', 'Client disconnected.', 'New Client.', 'Client disconnected.'. At the bottom, there are checkboxes for 'Autoscroll' (checked) and 'Show timestamp' (unchecked), and dropdown menus for 'No line ending' and '115200 baud', along with a 'Clear output' button.



4.1 ESP32 Web Server to Monitor Sensor Data

5. Demonstration

The demonstration video is shown below.



The screenshot shows a serial monitor window titled 'COM3' with a 'Send' button. The output text is as follows:

```
Connecting to MyWiFi
.....
WiFi connected.
IP address:
192.168.43.164
</ GET / HTTP/1.1
Host: 192.168.43.164
<b Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
Purpose: prefetch
Accept-Encoding: gzip, deflate
<
```

At the bottom of the window, there are checkboxes for 'Autoscroll' (checked) and 'Show timestamp' (unchecked), and dropdown menus for 'No line ending' and '115200 baud'. A 'Clear output' button is also present. Below the main window, a portion of another code editor is visible, showing a comment: '18 // Define timeout time in milliseconds (example: 2000ms)'.

4.2 Code Explanation

The Arduino code of this project is quite similar to the previous one. The main difference is in the HTML body where we send the potentiometer value (read from the pin A0) along with the HTML code in Line 82:

```
82      client.print(analogRead(A0)); // Read the potentiometer value
```

5. ESP32 as a Client to an IoT Cloud Platform

This section consists of

5.1 IoT Cloud Platform

5.2 Showing Sensor Data on ThingSpeak

5.3 Code Explanation

5.1 IoT Cloud Platforms: Connecting to the World

1) What are they?

- Specialized web services designed to ingest, store, visualize, and manage data from IoT devices.

2) Why use one?

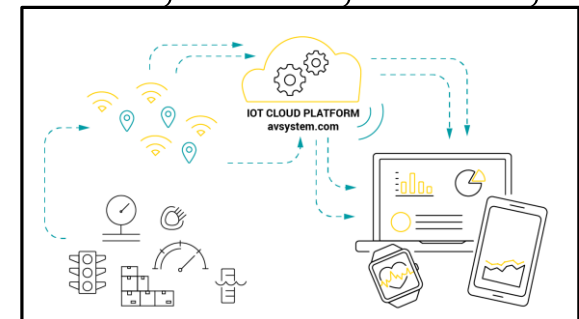
- It is the easiest way to monitor and control your devices securely over the internet, **from anywhere in the world.**

3) How do we connect?

- a) **The ESP32 acts as a client.**
- b) It sends data to the platform's API (Application Programming Interface), which is like a special, secure mailbox for your data.

4) Popular Platforms:

- ThingSpeak, Ubidot, Netpie (Nectec), Google, IBM Watson, AWS, Cisco, Kaa, Oracle, Thingworx, etc.



5.2 Sending ESP32 Data to the ThingSpeak Cloud

1. Project Outline

1) Project Goal:

- To send **sensor data** from an ESP32 to the **ThingSpeak cloud platform** for remote monitoring from anywhere.

2) Architecture:

- **Client:** ESP32 (reading a potentiometer on pin A0)
- **Server:** ThingSpeak IoT Cloud Platform

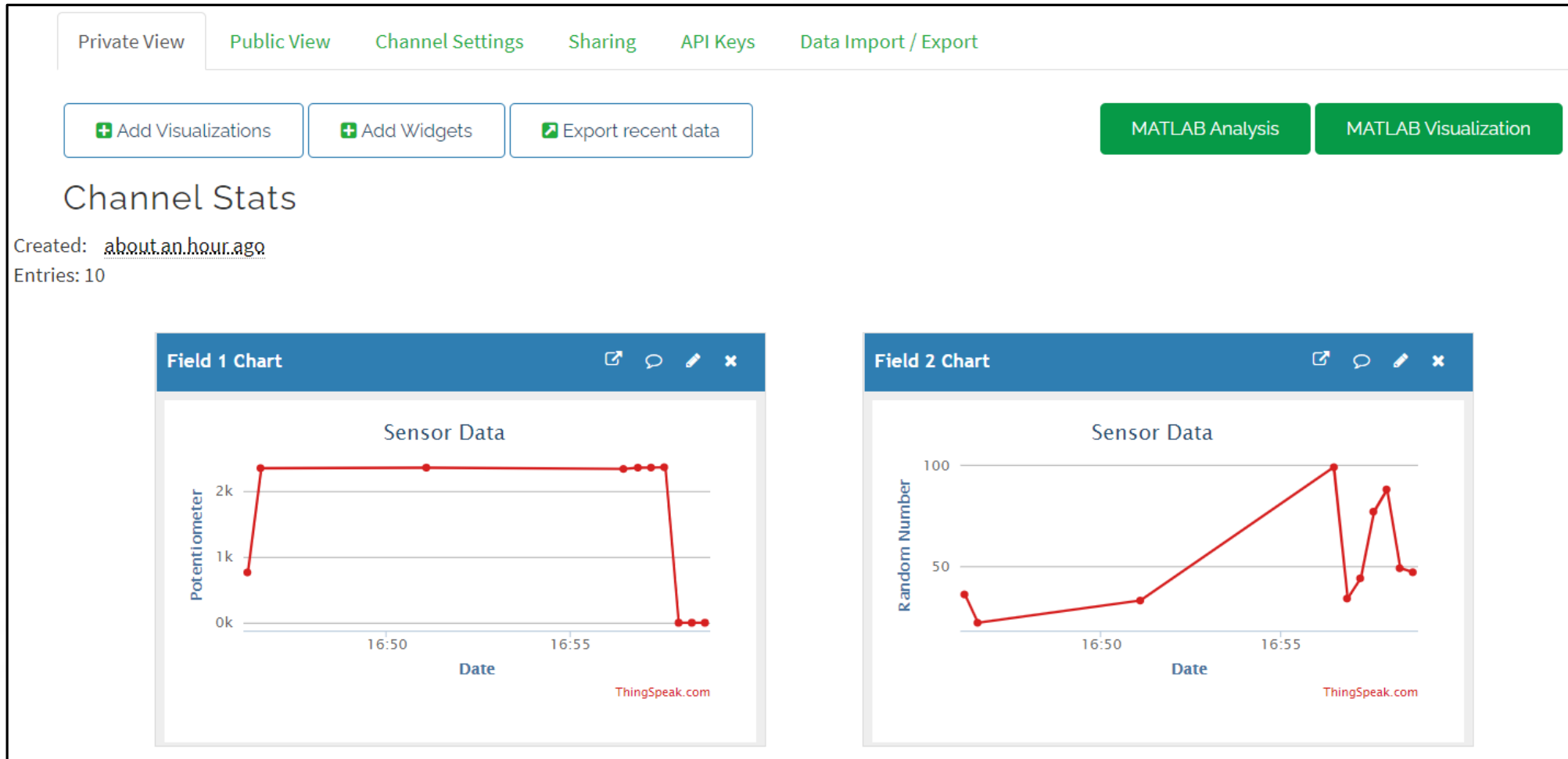
3) Process (Repeats every 10 seconds):

- Read:** The ESP32 reads the **current value from the potentiometer** and a **random number**.
- Send:** The ESP32 sends **two values** every 10 seconds via an HTTP Request to your unique ThingSpeak channel API.
- Visualize:** ThingSpeak receives these two values, stores them, and automatically plots them on two charts.

Result: A cloud-hosted dashboard that you can access from any browser to monitor your sensor data in real-time.

5.2 Sending ESP32 Data to the ThingSpeak Cloud

1. Project Outline



5.2 Sending ESP32 Data to the ThingSpeak Cloud

1. Project Outline



A Web Client

1) ESP32 send a HTTP request (potentiometer and a random number).



2) ThingSpeak sends back a HTTP response.



User using
a Web Browser

A Web Client

1) Browser sends a HTTP request.



2) ThingSpeak sends back a HTTP response including the HTML code.



ThingSpeak
Server

5.2 Sending ESP32 Data to the ThingSpeak Cloud

1. Project Outline

The key in this project is:

- 1) ESP32 sends the HTTP Get message to ThingSpeak. This message includes the **potentiometer value** and a **random number**.
- 2) The HTTP Get message is in **this format** (it is a string):

https://api.thingspeak.com/update?api_key=YourAPIkey&field1=250&field2=40

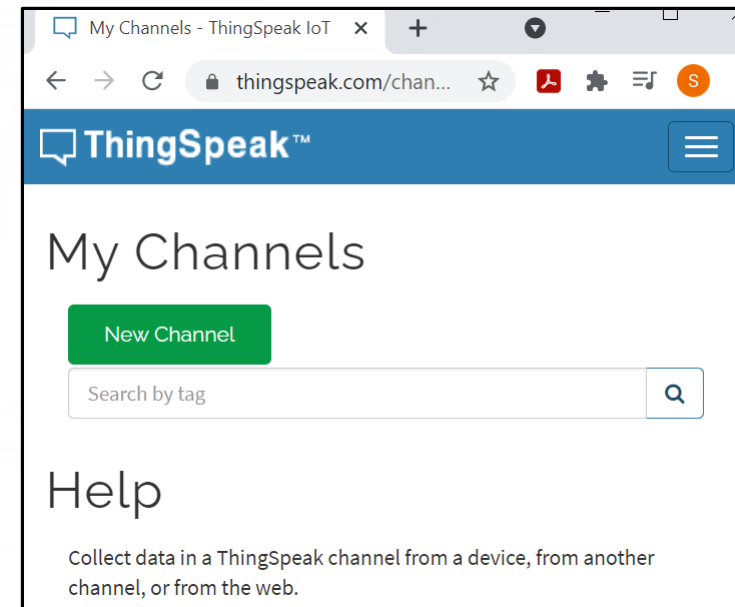
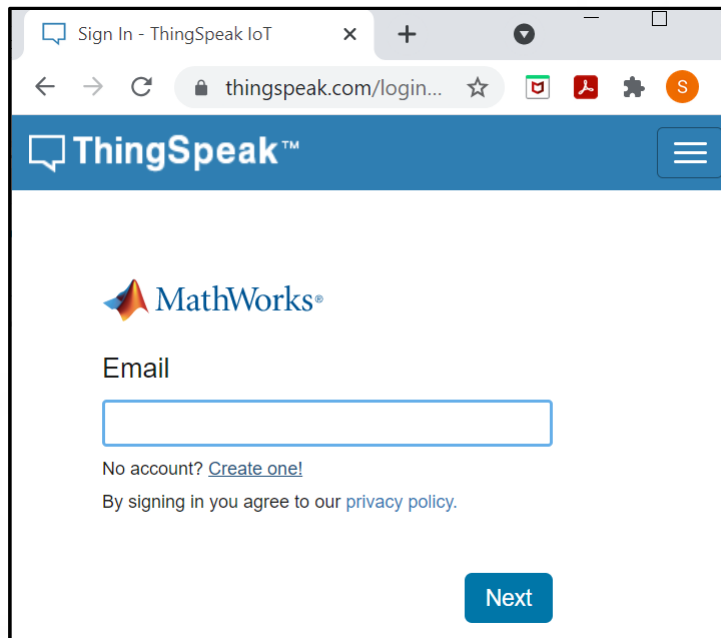
where YourAPIkey is your **ThingSpeak API key**, 250 is your potentiometer value, and 40 is your random number.

5.2 Sending ESP32 Data to the ThingSpeak Cloud

2. Sign up ThingSpeak and Create a Channel

ThingSpeak has a free API that allows you to store and retrieve data using HTTP. In this tutorial, you will use the ThingSpeak API to publish and visualize data in charts from anywhere. The following steps guide you to create a ThingSpeak channel.

- 1) Create a ThingSpeak account.
- 2) Create a new channel by clicking on the New Channel button.



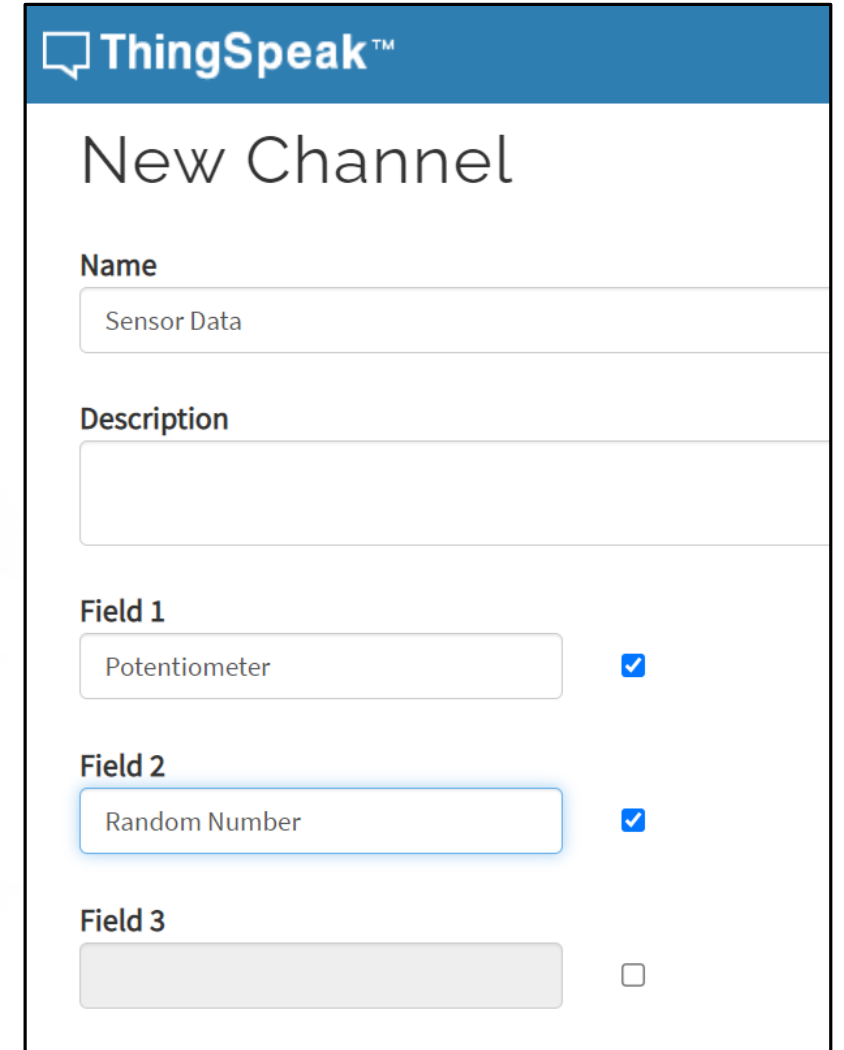
5.2 Sending ESP32 Data to the ThingSpeak Cloud

2. Sign up ThingSpeak and Create a Channel

- 3) In your new channel, fill in
- Name: Sensor Data
 - Field 1: Potentiometer
 - Field 2: Random Number

The other fields are blank. We fill in Field 1 and Field 2 since we will show two sensor data. Make sure that the boxes after Field 1 and Field 2 are checked.

- 4) Click the save button at the bottom of this page.



The screenshot shows the 'New Channel' page on the ThingSpeak website. The page has a blue header with the ThingSpeak logo. Below the header, the title 'New Channel' is displayed. There are three main input sections: 'Name', 'Description', and a list of fields. The 'Name' field contains 'Sensor Data'. The 'Description' field is empty. The 'Field 1' section has a text input 'Potentiometer' and a checked checkbox. The 'Field 2' section has a text input 'Random Number' and a checked checkbox. The 'Field 3' section has an empty text input and an unchecked checkbox.

ThingSpeak™

New Channel

Name

Sensor Data

Description

Field 1

Potentiometer ☒

Field 2

Random Number ☒

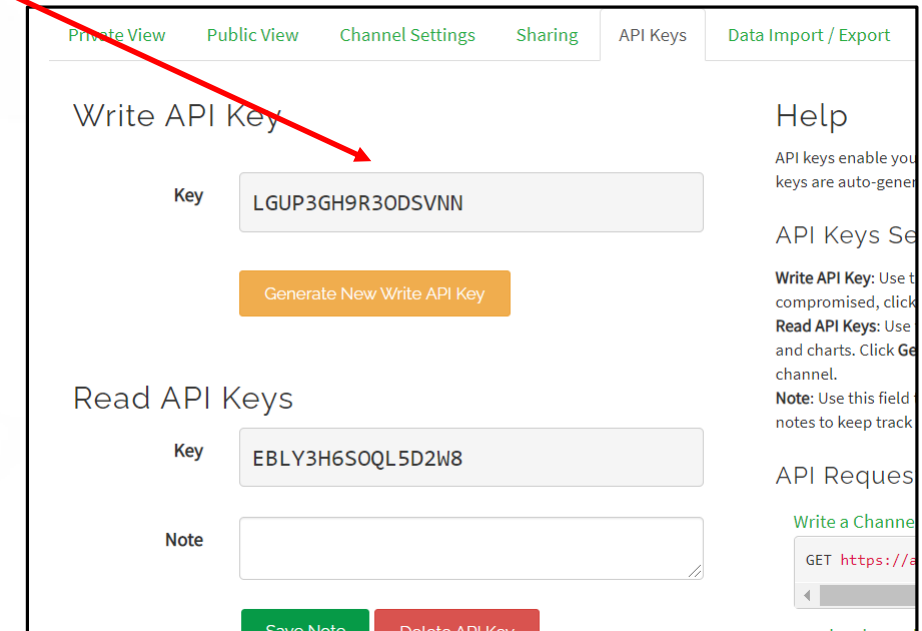
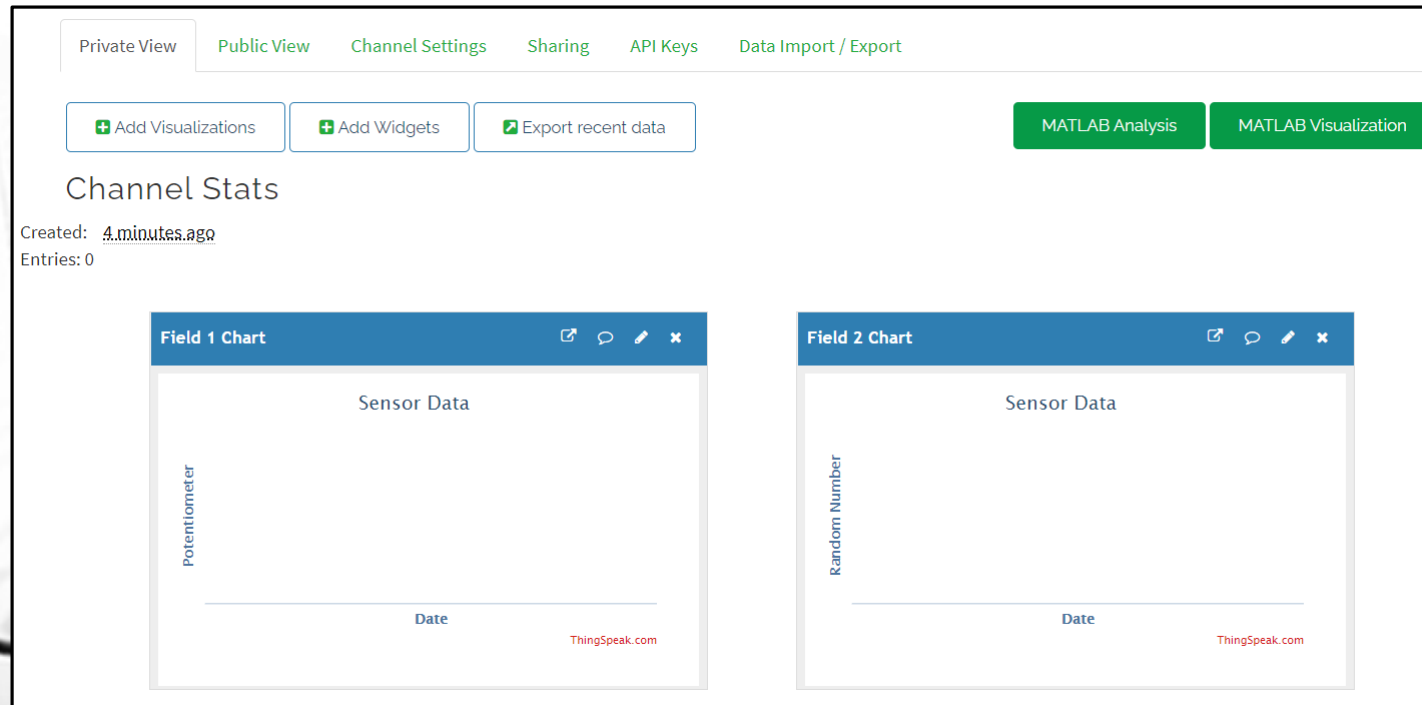
Field 3

☐

5.2 Sending ESP32 Data to the ThingSpeak Cloud

2. Sign up ThingSpeak and Create a Channel

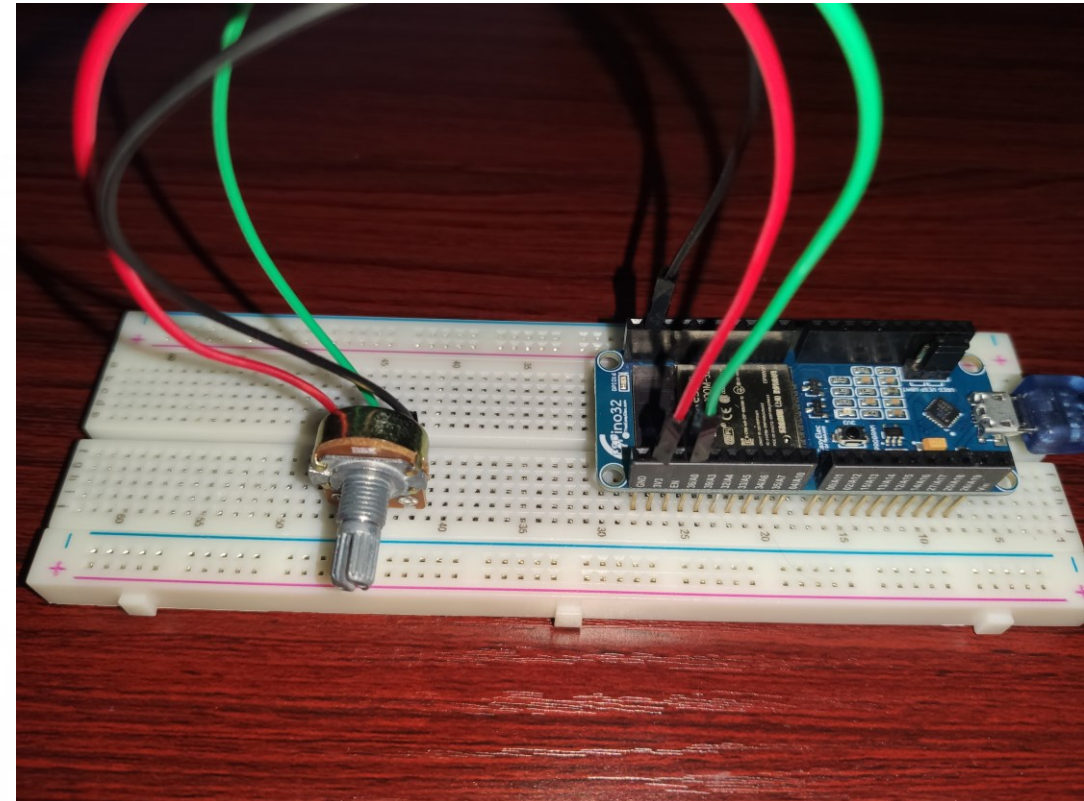
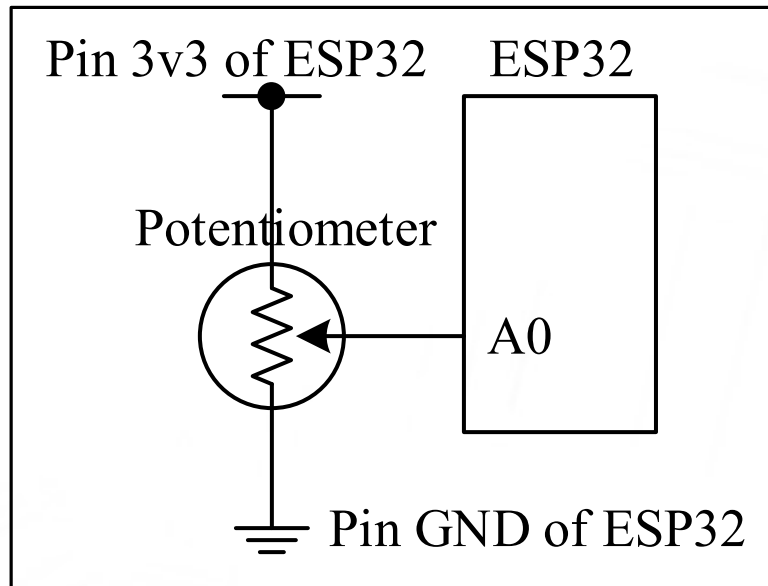
- 5) You will go to your ThingSpeak channel (Sensor Data). There are many tabs. When you click the private view tab, you will see the Filed 1 chart and Field 2 chart which will show the data sent from ESP32.
- 6) Click on the API Keys tab, to see your **Write API key**. Record it since we will use it in the Arduino code. Now, we are done.



5.2 Sending ESP32 Data to the ThingSpeak Cloud

3. Schematic

We connect a potentiometer to the pin A0 of ESP32 as shown in the following schematic diagram.



5.2 Sending ESP32 Data to the ThingSpeak Cloud

4. Arduino Code

The Arduino code “**WebClient_ThingSpeak.ino**” will ask **ESP32** to send two values (**potentiometer value and a random number**) to **ThingSpeak**. Please revise the code as follows and upload to your ESP32

- 1) Write this code in your Arduino IDE.
- 2) You have to fill in your SSID and Password of your access point.

```
4 // Replace with your network credentials
5 const char* ssid = "Your SSID";
6 const char* password = "Your Password";
```

- 3) Put your Write API Key from ThingSpeak.

```
8 // REPLACE WITH THINGSPEAK.COM API KEY
9 String serverName = "http://api.thingSpeak.com/update?api_key=YourWriteAPIkey";
```

- 4) Upload the code to ESP32

5.2 Sending ESP32 Data to the ThingSpeak Cloud

4. Arduino Code

```
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3
4 // Replace with your network credentials
5 const char* ssid = "MyWiFi";
6 const char* password = "Moon15Star18";
7
8 // REPLACE WITH THINGSPEAK.COM API KEY
9 String serverName = "http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN";
10 // EXAMPLE:
11 //String serverName = "http://api.thingspeak.com/update?api_key=7HQJM49R8JAPR";
12
13 // THE DEFAULT TIMER IS SET TO 10 SECONDS FOR TESTING PURPOSES
14 // For a final application, check the API call limits per hour/minute to avoid getting blocked/banned
15 unsigned long lastTime = 0;
16 // Set timer to 10 seconds (10000)
17 unsigned long timerDelay = 10000;
18
```


5.2 Sending ESP32 Data to the ThingSpeak Cloud

4. Arduino Code

```
19 void setup() {  
20     Serial.begin(115200);  
21     WiFi.begin(ssid, password);  
22     Serial.println("Connecting");  
23     while(WiFi.status() != WL_CONNECTED) {  
24         delay(500);  
25         Serial.print(".");  
26     }  
27     Serial.println("");  
28     Serial.print("Connected to WiFi network with IP Address: ");  
29     Serial.println(WiFi.localIP());  
30     Serial.println("Timer set to 10 seconds (timerDelay variable), it will take 10 seconds before publishing the  
31 }  
32
```

5.2 Sending ESP32 Data to the ThingSpeak Cloud

4. Arduino Code

```
33 void loop() {  
34     // Send an HTTP GET request  
35     if ((millis() - lastTime) > timerDelay) {  
36         // Check WiFi connection status  
37         if(WiFi.status()== WL_CONNECTED){  
38             WiFiClient client;  
39             HTTPClient http;  
40             String HttpRequest = serverName + "&field1=" + String(analogRead(A0)) + "&field2=" + String(random(100));  
41  
42             // Create the HTTP Get request to ThingSpeak  
43             Serial.print("HTTP Request: ");  
44             Serial.println(HttpRequest);  
45             http.begin(client, HttpRequest);  
46             //Another option --> http.begin(client, HttpRequest.c_str());  
47         }
```

5.2 Sending ESP32 Data to the ThingSpeak Cloud

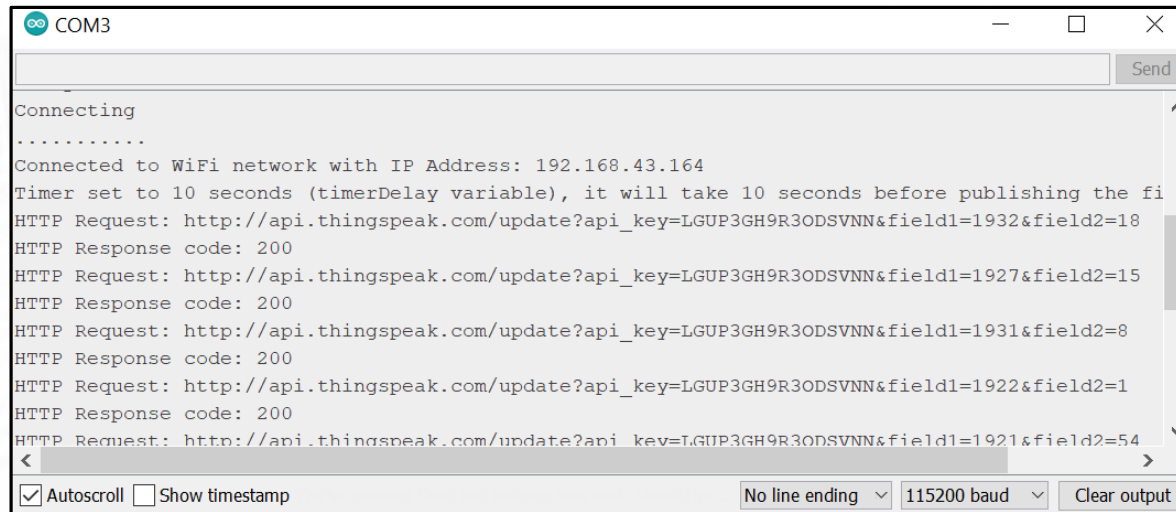
4. Arduino Code

```
48 // Send HTTP GET request
49 int httpResponseCode = http.GET();
50 if (httpResponseCode>0) {
51     Serial.print("HTTP Response code: ");
52     Serial.println(httpResponseCode);
53 }
54 else {
55     Serial.print("Error code: ");
56     Serial.println(httpResponseCode);
57 }
58 // Free resources
59 http.end();
60 }
61 else {
62     Serial.println("WiFi Disconnected");
63 }
64 lastTime = millis();
65 }
66 }
```

5.2 Sending ESP32 Data to the ThingSpeak Cloud

5. Demonstration

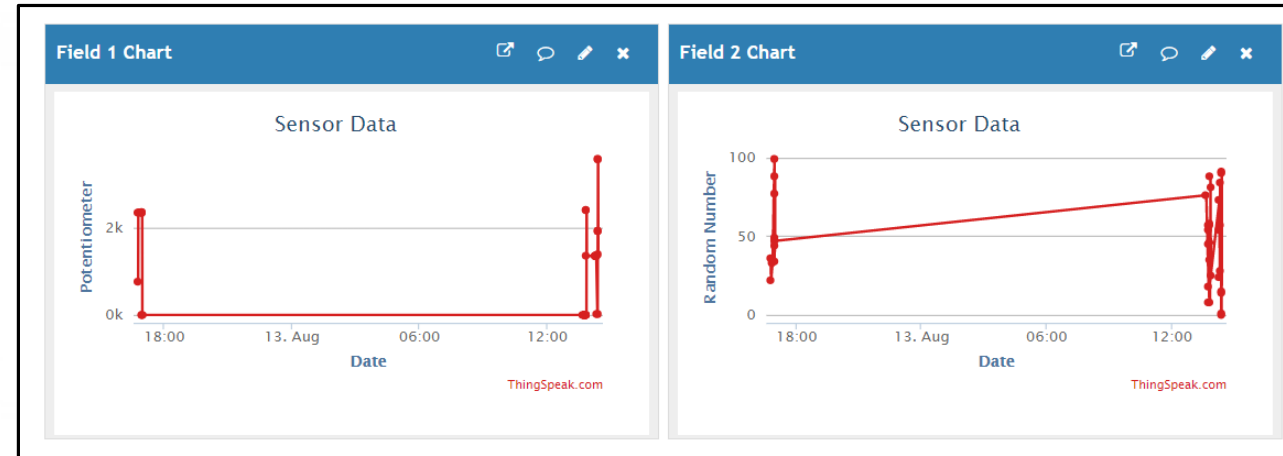
- 1) When finishing uploading the code to ESP32, ESP32 will try to connect the WiFi access point.
- 2) ESP32 sends the HTTP request message to ThingSpeak every 10 seconds.
- 3) ThingSpeak receives the message, send a HTTP response back, and update the charts.



COM3

Connecting
.....
Connected to WiFi network with IP Address: 192.168.43.164
Timer set to 10 seconds (timerDelay variable), it will take 10 seconds before publishing the fi
HTTP Request: http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN&field1=1932&field2=18
HTTP Response code: 200
HTTP Request: http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN&field1=1927&field2=15
HTTP Response code: 200
HTTP Request: http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN&field1=1931&field2=8
HTTP Response code: 200
HTTP Request: http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN&field1=1922&field2=1
HTTP Response code: 200
HTTP Request: http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN&field1=1921&field2=54

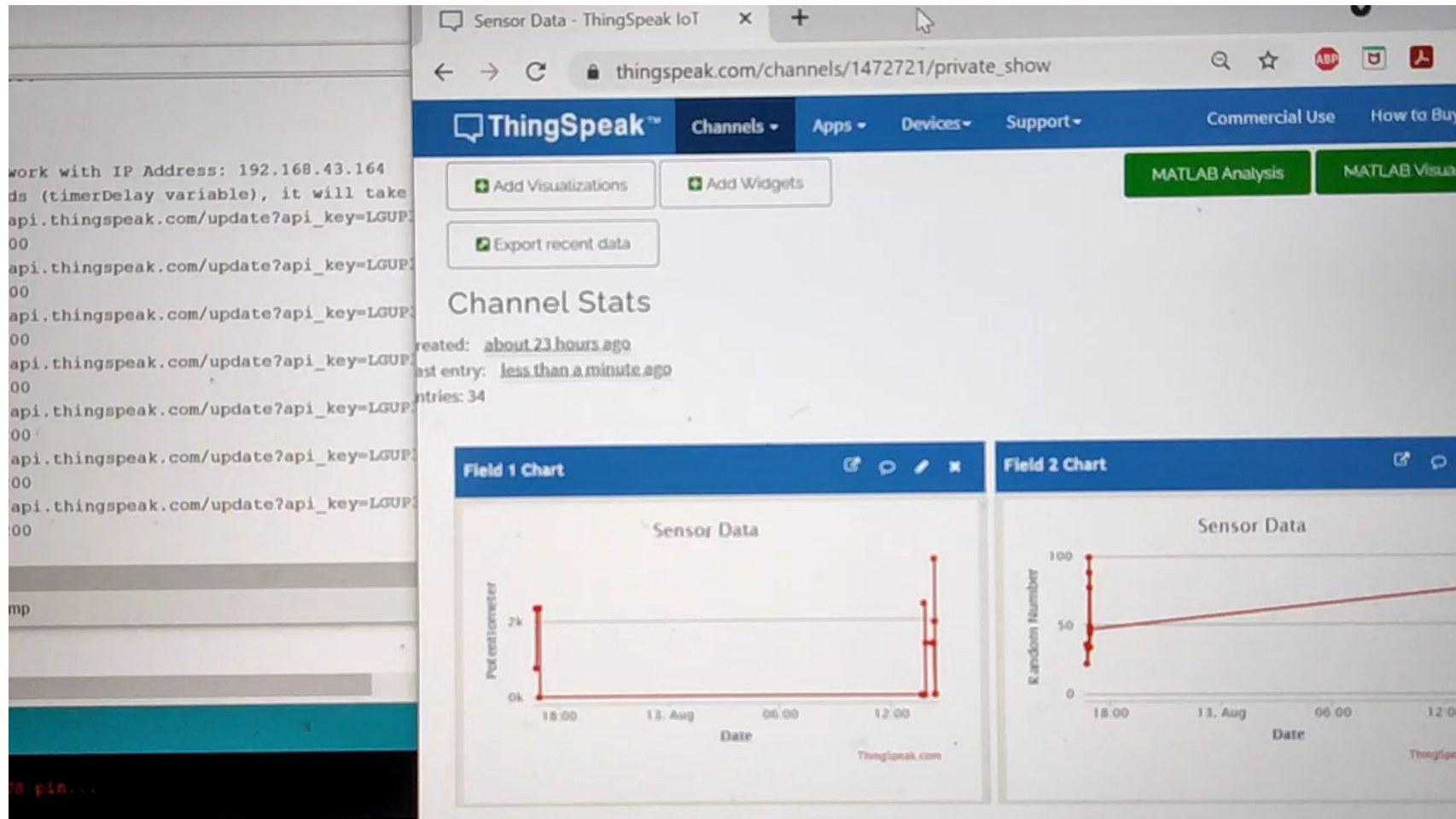
☒ Autoscroll ☐ Show timestamp



5.2 Sending ESP32 Data to the ThingSpeak Cloud

5. Demonstration

A demonstration video is shown below.



5.3 Code Explanation

Mainly the code consists of the following parts.

- 1) Specify your ThingSpeak API key.

```
8 // REPLACE WITH THINGSPEAK.COM API KEY
9 String serverName = "http://api.thingspeak.com/update?api_key=LGUP3GH9R3ODSVNN";
```

- 2) Set ESP32 as a WiFi client and a HTTP client (to be able to send a HTTP Get request).

```
38 WiFiClient client;
39 HTTPClient http;
```

- 3) Create a HTTP request message.

```
40 String HttpRequest = serverName + "&field1=" + String(analogRead(A0)) + "&field2=" + String(random(100));
```

5.3 Code Explanation

- 4) Call the begin method on the http object to store this URL on the http object.

```
45      http.begin(client, HttpRequest);
```

- 5) Send the HTTP request by the GET method.

```
49      int httpResponseCode = http.GET();
```

where httpResponseCode stores the returned code from ThingSpeak.

6. Advanced Topics for ESP32 Web Servers and Clients

Advanced Topics for ESP32 Web Servers

The following topics could be useful for your work in the future.

1. You might create a password to log into the ESP32 web server:
<https://randomnerdtutorials.com/esp32-esp8266-web-server-http-authentication/>
2. You might consider the web socket approach to automatically update the data on your web page:
<https://randomnerdtutorials.com/esp32-websocket-server-arduino/>
3. You might consider the asynchronous web server approach such that no information is shown in the URL of the web browser (which is not secure):
<https://randomnerdtutorials.com/esp32-async-web-server-espasyncwebserver-library/>
4. You might implement your ESP32 web to access from anywhere (globally):
<https://microcontrollerslab.com/accessing-esp32-web-server-anywhere-world-esp8266/>

Advanced Topics for ESP32 Web Clients

The following topics could be useful for your work in the future.

1. You might find more examples to use the HTTP GET method:
<https://randomnerdtutorials.com/esp32-http-get-open-weather-map-thingspeak-arduino/>
2. You might find more examples to use the HTTP POST method:
<https://randomnerdtutorials.com/esp32-http-post-ifttt-thingspeak-arduino/>
3. You might set ESP32 to send an email:
<https://randomnerdtutorials.com/esp32-send-email-smtp-server-arduino-ide/>
4. You might send your data to other IoT cloud platforms such as Google, AWS, etc.

Questions?